

THESIS

Tanui Sila Kipng'etich

Debrecen
2026

UNIVERSITY OF DEBRECEN
FACULTY OF INFORMATICS
DEPARTMENT OF INFORMATION TECHNOLOGY

**AI-DRIVEN E-COMMERCE AUTOMATION USING BLAZOR
WEB FRAMEWORK**

Supervisor:

Dr. Bérczes Tamás Márton
Professor, Department of Informatics
Systems and Networks

Candidate:

Tanui Sila Kipng'etich
MSc. Computer Science
Neptun Code: P8J80D

Debrecen,
2026

STATEMENT OF INDIVIDUAL WORK

I, Tanui Sila Kipng'etich, Neptun code: P8J80D, a student of MSc Computer Science program at the University of Debrecen, Faculty of Informatics, declare that this thesis is my own work. I have not used any other sources or aids than those indicated. Every part of the thesis that has been taken from other works, either literally or corresponding to the sense, is clearly marked and provided with the respective citation. I used the specified help of the supervisors for the technical and formal realization of thesis.

Debrecen, 2026.04.07

Signature

ACKNOWLEDGEMENTS

I would like to take this moment to like a supervisor, Dr. Bérczes Tamás Márton, for the guidance Technical and professional that he gave me while I was doing this thesis, I was motivated by his work during Cloud Computing unit. He was my greatest motivation to do a thesis that is related to Cloud Computing.

Secondly, I would like to thank the faculty of informatics and all my lecturers who gave me the knowledge that has aided me in the development of this thesis. I want to thank them for the academic environment for the resources there were necessary during this period.

I would also want to give a special shoutout to the Transcosmos and the Fossil Group. It was my work here that I gained more information that was essential during the development of my horology system and gave me a lot of understanding about the E-commerce and luxury industry. Also, it gave me a lot of knowledge about the requirements that are needed for a real-world E-commerce website.

Lastly, I wish to thank my family and friends for their support, moral, and financial encouragement during my entire studies.

Table of Contents

STATEMENT OF INDIVIDUAL WORK	i
ACKNOWLEDGEMENTS	ii
CHAPTER 1: INTRODUCTION.....	1
1.1. Background of Study	1
1.2. Personal Context and Professional Background.....	2
1.3. Problem Statement.....	3
1.3.1. <i>The Crisis of Knowledge Asymmetry</i>	3
1.3.2. <i>Operational Scalability and Response Latency</i>	4
1.3.3. <i>Regional Payment Friction in Emerging Markets</i>	4
1.4. Motivation	5
1.5. Research Objectives	6
1.6. Research Questions	6
1.7. Scope and Delimitations	7
1.8. Definition of Terms	7
1.9. Significance of Study	7
2.1. Introduction to the Theoretical Framework.....	9
2.2. Evolution of Web Application Architectures	9
2.3. The Rise of AI Agents in E-Commerce	10
2.4. Retrieval-Augmented Generation (RAG) Architecture.....	11
2.5. Vector Databases and Semantic Search.....	12
2.6. Localized Digital Payments in Emerging Markets.....	13
2.7. Summary and Synthesis	14

CHAPTER 3: SYSTEM DESIGN & ARCHITECTURE	16
3.1. General System Architecture	16
3.1.1. <i>The N-Tier Layered Pattern</i>	16
3.1.2. <i>The Repository and Unit of Work Patterns</i>	17
3.2. Database and Persistence Engineering	17
3.2.1. <i>Relational Schema Deep-Dive (SQLite/EF Core)</i>	17
3.2.2. <i>Chatbot Persistence Logic</i>	18
3.2.3. <i>Non-Relational Vector Storage (Qdrant Topology)</i>	20
3.2.4. <i>Multi-Model Data Orchestration</i>	21
3.3. AI Agent Architectural Framework (RAG Logic)	23
3.3.1. <i>The Embedding Pipeline</i>	23
3.4. Payment Integration Architecture (M-PESA/Daraja)	25
3.4.1. <i>The M-PESA STK Push Handshake</i>	25
3.5. Security and Identity Framework	27
3.5.1. <i>Token-Based Authentication Flow</i>	27
3.6. Real-time Infrastructure and SignalR	29
3.7. Deployment Strategy and Cloud Infrastructure	31
Chapter 4: Implementation and Results	33
4.1. Development Environment and Tools	33
4.2. Infrastructure and Deployment Setup	34
4.3.1. <i>Product Catalog, Cart, and Checkout</i>	35
4.3.2. <i>RAG Chatbot Implementation Notes</i>	39
4.3.3. <i>M-PESA Integration Implementation Notes</i>	40
4.3.4. <i>Admin Dashboard</i>	42
4.4. Testing and Validation	46

4.4.1. <i>Functional Testing</i>	46
4.4.2. <i>Security Validation</i>	47
4.5. Performance Observations.....	48
4.6. Summary.....	48
Chapter 5: Conclusion	50
5.1. Summary of Work	50
5.2. Research Objectives Revisited	51
5.3. Research Questions Answered	52
5.3.1. <i>How can RAG be optimized for factual accuracy in a technical retail context?</i>	52
5.3.2. <i>Does M-PESA integration reduce cart abandonment for Kenyan shoppers?</i>	53
5.3.3. <i>What are the trade-offs of Blazor WebAssembly compared to JavaScript frameworks for AI-heavy applications?</i>	53
5.4. Limitations of the Study	54
5.5. Future Work and Recommendations	55
5.6. Final Reflection	56
Bibliography	58
DECLARATION ON THE USE OF AI	60

CHAPTER 1: INTRODUCTION

1.1. Background of Study

There has been a major shift in regard to the e-commerce sector, especially for luxury businesses. Businesses nowadays have shifted from physical stores to digital e-commerce stores. This has helped businesses to achieve 24/7 continuation of business and also break geographical boundaries. This transformation has been accelerated by the improvement of internet accessibility and by the change in consumer behavior; this has made industries change how they engage their customers. As we talk about the luxury retail sector—for this case we will be dealing with horology—the shift has brought about a unique challenge: how to improve exclusivity and ensure the customers are able to understand each and every detail of what they want to order. According to McKinsey & Company (2024), luxury e-commerce is no longer a secondary channel, but a primary touchpoint for more than 70% of Gen Z and millennial clients.

In the past, purchasing a high-end watch, was an event that took place with the guidance of watchmaking experts in exclusive boutiques. These experts served as a bridge between the mechanical complexity of the movement and the buyer's aesthetic preferences. In today's digital age, there is a brand that is sought and broken, where consumers are presented with big amounts of data, but most of the time they lack the context and expertise to interpret it. As Nielsen (2023) points out, he talked about the Paradox of Choice and how it leads to decision-making paralysis, which has caused a high rate of shopping cart abandonment in tech retail.

As artificial intelligence is becoming more common nowadays, the question is: how can we integrate this invention into our modern web frameworks, like Microsoft Blazor, so that it can provide expert and informed decisions which are especially capable of assisting in technical fields at scale (Deloitte, 2024).

In addition, we need to talk about how we can integrate technologies that will enable us to bridge regional disparities, especially regarding payment technology. The global North is fortunate to have a standardized way of making orders by using card payments. But how do regions which aren't as technologically advanced become able to make online payments and reduce cart abandonment in

e-commerce websites? Take, for example, countries in emerging economies, like those in East Africa; countries like Kenya have been fortunate to pioneer the use of the mobile money ecosystem like M-Pesa.

To say that an e-commerce platform is truly automated and effective, it should be able to be technically intelligent and also factor in cultural or regional compatibility. These things will form the core architectural motivation for this research.

1.2. Personal Context and Professional Background

The motivation to do this research was influenced by my tenure working as a customer care agent at Transcosmos, Hungary. I worked for a luxury company called Fossil, and this is where I came to understand how to approach customers, how to ensure that I give them the correct information, and how to solve most of the problems they had. On a daily basis, I acted as a human main point of contact when customers asked questions regarding product catalogs, and I was able to answer based on the information that I had been given during training.

But sometimes, you meet a customer who is knowledgeable and demanding, and without enough information, you find yourself in a systemic breakdown. I remember multiple scenarios where a customer would want to know exactly about a specific feature in a watch, like the material of the balance spring or the frequency of the movement. As much as I tried to answer those particular questions, it took a lot of time to inquire from the relevant departments and internal databases to understand the specific features of the watch. This caused what we call a supervisor bottleneck, where as an agent, we had to wait for higher-level intervention or documentation clarification. This led to a wait time of over 15 minutes for a single technical question.

While working on the front lines, I came to realize that most of the customers were frustrated not because we lacked a solution, but because of the time it took to provide the much-needed information to them. When we talk about the luxury market, every minute can cost your company and cause the brand to degrade.

Most of the questions asked by our clients were repetitive; however, we took a lot of time to process this information by searching in outdated and disorganized knowledge-based systems, which wasted both our time and the customer's time. This brought about my realization—or what I'd term

a "Eureka moment": what if we had given a machine all the required information that an expert watchmaker would have? We could have saved the company a lot of resources as well as improved the company brand.

Because of this background as a customer care agent, it gave me an opportunity to provide a user-centric perspective, which is always missing when we do research usually led by pure architectural software research. It made me learn that an e-commerce platform should not just be about the correct CRUD operations, but rather should be about conversation. I came to realize that there are three areas of failure in the manual systems:

- Human agents are exhausted when there is a high volume of pressure.
- There is usually high inconsistency regarding the answers provided when agents are working across different shifts.
- The cost and the time required to train staff to master the details required by those luxury premium brands.

These three points are what directed me as I was proposing the Tanui Watch Store system.

1.3. Problem Statement

Although the e-commerce sector has matured, the retail industry for high-tech, luxury products is now facing three interconnected challenges that cannot be solved on a large scale or effectively through business models, Human-centric business models cannot resolve these challenges on a large-scale or effective basis. These challenges pose a direct threat to the sustainability of luxury brands that have adopted a “digital-first” approach.

1.3.1. The Crisis of Knowledge Asymmetry

Think of this scenario where a customer asks a particular question, and for this case, we will use an example of a horology store. They ask a question regarding internal calibers, silicon components, or the water resistance rating of a particular watch. But the answer they get back is simple and generic, or they don't have the mechanism for getting the best answers because the site usually uses filters or search results. This may make the customer frustrated because they may lack the enough information needed to purchase a particular product.

According to Smith and Jones (2022), they stated that when a customer, and for that case, in a luxury segment, receives a simple, generic answer when they required a specific mechanical question to be addressed, and that particular question is not addressed, they lose interest in the product. This therefore causes the brand to lose its value even though the price isn't the issue. This may create a situation where the company has to spend a lot of money and resources to facilitate returns of items that the customer sent back due to inadequate information about that particular product.

The gap of our knowledge asymmetry is usually caused by technical complexity that doesn't go hand in hand with the available information. The customer feels disconnected by the amount of information being provided by the website or the agents handling their queries.

1.3.2. Operational Scalability and Response Latency

I worked as a customer care agent and I noticed in modern e-commerce there are a lot of customers who reach out to ask a specific query. These customers may come in thousands and thousands, and they come with different requests at the same time. If we depend on a people-centric model where we hire agents to handle those queries, then as more queries are sent by the customers, we have to hire more people and train them for quite a long time, especially in the luxury market, to ensure that the information provided is as accurate as possible.

This would cost the company a significant amount of money and resources. According to Forrester Research (2024), they noted that 55% of customers have a likelihood of abandoning their carts during online shopping when they ask a question but do not receive a quick answer. If this happens, then it will cost the company a lot. Think of promotional periods like Black Friday or when there is a new product that is launched by the company; there are a lot of questions raised by the customers, and if there is not enough customer support to answer those queries, then the company will face challenges and may lose customers in the process.

1.3.3. Regional Payment Friction in Emerging Markets

The third point is regional payment friction in emerging markets. Having stayed in the global North, the primary payment method is by using Mastercard, Visa, or PayPal; but in a growing market like Kenya, where this thesis will focus, more than 90% of the Kenyan market relies heavily

on M-Pesa as a method of payment, and most of the Kenyan market do not have to open a bank account to transact. So, using the global North kind of payment methods won't work well for the Kenyan market.

To break this payment friction, I will be incorporating the use of M-Pesa, which will be provided by the Daraja API. This will enable me to consume the data that will be sent and received from M-Pesa. According to Kiwuitu (2023), he noted that most Kenyan customers abandon their carts, especially at the checkout page, because they are not getting a preferable method of payment.

1.4. Motivation

The motivation behind this research is to combine my professional experience as a customer support agent with modern software engineering principles in the current e-commerce market. My goal is to improve how customers search for products and ensure they receive answers from a website without long wait times, specifically by improving their conversational experience with chatbots.

The motivation behind this research is how I can combine my professional experience as a customer support agent in a luxury business together with the knowledge that I have gained over the period of my studies. The goal of this thesis is to improve how customers get information regarding products, especially in the luxury e-commerce setting, without having to wait for long periods of time. Also, I would like to add a human touch to the way chatbots answer customers.

To achieve this, I'll be using Retrieval-Augmented Generation (RAG) technology, which will be integrated to enable the error-free artificial intelligence that is required, especially in the luxury e-commerce sector. This is driven by the fact that for a luxury business, accurate information is vital. The goal of this thesis is to ensure that we create a system that behaves more like a human and answers more like a human but also taking into consideration the amount of time that is taken to answer a customer. Simple questions like "Where is my order?" or "Is this available in blue?" or any other technical questions will be able to be answered within the shortest time possible with the integration of a human touch, while also ensuring that there is utmost accuracy in the answers provided.

But this raises a question: do we still require humans to work on the front lines? I still believe that human intervention is very important, but there are some questions that can be easily answered without human intervention. This will save the company a lot of time by answering customers with repetitive questions, or those questions that do not require a human to answer them. Although human intervention is necessary, there are some questions that require a lot of time to find out the truth about a particular product. So, my aim is to create a system that answers those questions effectively and leaves humans to handle the questions that require human intervention.

1.5. Research Objectives

The primary objective of this particular thesis is to provide a comprehensive AI-integrated store that would be able to solve human queries within the shortest time possible with utmost accuracy. Specific objectives include:

- To design a front-end architecture with the help of Blazor WebAssembly that will provide a near-native experience and enable customers to browse.
- To implement a Retrieval-Augmented Generation (RAG) pipeline, which will transform technical documentation into an easily searchable vector space by leveraging OpenAI embedding models.
- To build a .NET-based back end that enables a seamless flow of information between a SQL relational database (for example, for orders) and a Qdrant vector search for technical product knowledge.
- To integrate the Safaricom Daraja API 2.0 to support one-click mobile payments by leveraging STK Push functionality.
- To validate the system and ensure that it reduces query latency compared to human customer care agents.

1.6. Research Questions

- How can the RAG architecture be optimized to ensure 100% factual grounding in high-stakes technical product environments like horology?
- Does the integration of localized mobile payment methods significantly decrease the 'Abandoned Cart' rate in the Kenyan luxury demographic?

- What are the performance trade-offs of using C# and Blazor WebAssembly for high-frequency AI interactions compared to traditional JavaScript frameworks like React?

1.7. Scope and Delimitations

This research will focus on how we can implement an e-commerce platform for luxury watches, which I'll call the Tanui Watch Store. The focus of this is to enable an AI model that answers exactly like a human and is as creative as a human.

I'll make use of the payment method called M-Pesa, which uses an STK push protocol that will be provided by the Safaricom Daraja Gateway. My focus will be for those customers who have a stable internet connection for WebAssembly execution, but I will not be able to cover offline-first scenarios. My geographical market will be Kenya, and I'll try as much as possible to localize my features while also ensuring that they adhere to the global standards of luxury e-commerce.

1.8. Definition of Terms

Blazor: A web framework by Microsoft for building interactive web UIs using C# instead of JavaScript.

RAG (Retrieval-Augmented Generation): A technique for enhancing the output of an LLM with external facts retrieved from a trusted knowledge source.

Vector Database: A storage system optimized for mathematical multi-dimensional representations of data.

Daraja API: This is a developer gateway that allows businesses to access Safaricom's M-PESA API services.

Horological Complication: This is any feature of a watch ranging from the simple display of hours, minutes, seconds, etc.

1.9. Significance of Study

This study will be very important especially for three distinct stakeholder groups:

- **Retailers:** It will enable them to enormously reduce operational costs and improve how customers are engaged.
- **Clients:** It will provide them with expert, 24/7 answers to their technical queries, which at the end improve their shopping experience and satisfaction.

- **The Academic Community:** This thesis will demonstrate the practicality of the use of Retrieval-Augmented Generation (RAG) together with .NET and Blazor technology.

CHAPTER 2: LITERATURE REVIEW

2.1. Introduction to the Theoretical Framework

When building an AI-powered e-commerce platform, some considerations are needed in the technical domain to be implemented while developing the website. When I am talking about the technical domain, this may range from the use of modern web systems like distributed systems, or basically the integration of natural language processing technologies like the OpenAI model, as it will be used in this particular thesis. This will allow a smooth transition of knowledge and incorporate a human touch to automate some of the most repetitive tasks that are usually done by customer care agents. These tasks are those that usually require human intervention, but as technology evolves, we need to ensure they can be automated. We can save resources and time so that customer care agents can focus on those tasks that are more technical or difficult and require critical thinking that can only be done by human beings.

To implement this thesis, the major focus is how we can use current technologies like the Microsoft Blazor framework and Retrieval-Augmented Generation to automate some of the tasks that human customer care agents usually perform.

Another thing to think about is how countries in the Global South, for example, Kenya, transact or make payments. Do they pay using cards or PayPal? To be able to answer this particular question, and after digging, I came to realize that most African countries are not exposed much to those technologies. And this doesn't mean that they do not have their own technology; mobile payment methods have become very, very common. In Kenya, there is a technological innovation called M-PESA, which is a fast and secure method of payment. It has been there for nearly 20 years now, and it outperforms most technologies that have been introduced. This thesis will try to incorporate that idea, and together with RAG and the Microsoft Blazor framework, it will help us to see how much this will improve customer satisfaction and reduce the tendency of cart abandonment, especially in the checkout of websites.

2.2. Evolution of Web Application Architectures

There has been a constant struggle regarding how we can control responsiveness through the integration of the server-side and the client side. When we think about traditional architectures like

ASP.NET MVC, which relied heavily on full-page post backs, these have been realized to cause significant latency, especially in interactive applications. With the rise of JavaScript-heavy Single Page Applications (SPAs), taking for example React and Angular, the issue of latency has been addressed, but it also introduced the idea of context switching overhead, where developers can maintain duplicate models and logic in different languages (Anderson, 2022).

Microsoft's introduction of the Blazor framework has presented us with a major shift in this particular space. This thesis will utilize Microsoft WebAssembly (WASM) and Blazor, which allow us to compile C# code and enable it to run directly in the browser sandbox at almost near-native speed. According to Microsoft (2024) documentation, the primary advantage of Blazor WebAssembly in an e-commerce context is the ability to share Data Transfer Objects (DTOs) and improve validation logic between the client and the server. This isomorphic development model helps us reduce synchronization errors and, hence, accelerates the development of data-intensive features, such as real-time inventory tracking and AI chat interfaces.

2.3. The Rise of AI Agents in E-Commerce

Every day, there is an improvement in the use of AI, especially in the retail world. We moved from simple decision trees, which were generally unable to handle queries effectively, to a need for large language models that can answer queries, especially from those buyers who require a lot of information about a particular item. With this, we need to find the best way to use AI to answer those queries. According to Gartner (2024), he said that we are now in an era where we do not just use these models to complete simple tasks, but we need to use them to improve the reasoning and execution of difficult tasks. We are at a moment where these agents are intelligent enough and capable of translating questions regarding mechanical specifications, or any other questions that would be difficult for normal customer care agents to answer.

However, by directly adopting these large language models in the luxury sector, we face a significant risk. We understand that in a luxury business, the authenticity of information is key; any false information would cost the company a lot. By incorporating an LLM directly into our e-commerce websites, we are faced with the issue of hallucination. This is basically where the model fails to consider the correctness of the information provided; it just picks answers from any source

and throws them at the user. Now, the question is: what if we restrain this model to only answer questions based on the data we have in our systems, without allowing it to use any other available online answers to address the customer's query?

To achieve this, we want to train our model to read from the specific data in our database and answer like a human. By doing this, we improve the authenticity and correctness of the answer for every question asked by our clients. This aligns with Nielsen (2023), who coined the phrase Grounded AI, which strictly constrains our LLM within our own knowledge base without using external knowledge.

2.4. Retrieval-Augmented Generation (RAG) Architecture

To be able to achieve the idea behind Tanui customer support, the idea of Retrieval-Augmented Generation is very essential. And what is RAG? It is essentially a way of fetching information from local storage, like company documents or a company database, and using that information to answer a customer's query. To do this, we rely completely on the detailed company information rather than using the model's already pre-trained data.

In a study done by Lewis et al. (2020), it was shown that a model has the capability of pulling and extracting the right information if it is given a chance to read from the databases and documents before it answers a customer's query. This will enable accuracy, especially when addressing technical topics which a human customer care agent cannot answer accurately or would require a lot of time to dig into their documents to get the right information. And as this thesis is focusing on watch e-commerce, we need to ensure that any information that we are going to give to the user or the customer is as accurate as possible. We don't want to give wrong information to our customers, which would ruin our reputation and lead to poor customer support. Correctness of information should be paramount, especially when we are dealing with questions regarding our watch catalogue and their technical specifications, as this may not have been provided to the model during its initial training.

Looking at the RAG process, it consists of six key steps:

1. **User query:** This is basically any question asked by a customer.

2. **Text embedding:** This is where we convert the user's query into a numerical format; this will be discussed later.
3. **Vector Database:** This enables our system to be able to search directly through documents and databases within our company's data.
4. **Context augmentation:** Having retrieved the documents and relevant information, we then add this information to the user's original query to provide an accurate baseline for answering the customer's questions.
5. **Large language model:** This will enable us to give the human-like answers to their queries in a more contextual manner by combining the query and the context.
6. **Factual AI response:** This is the last step where the system generates an understandable answer for the customer.

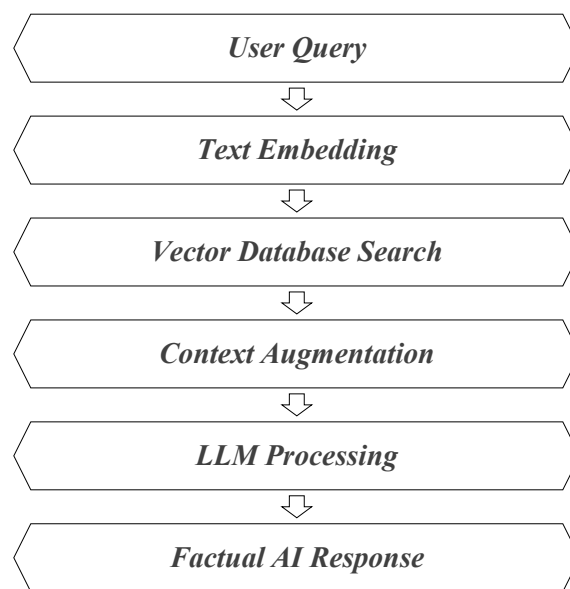


Figure 2.1: Diagram showing the 6 steps of RAG pipeline.

2.5. Vector Databases and Semantic Search

The next idea is the use of vector databases and semantic search. We have been using traditional relational databases, and they're usually pretty good when tackling structured queries—like a `SELECT` to find a specific order ID or an `UPDATE` to change a price. But for items to be found directly, there is a need to think about synonyms or the similarity in meaning. Think of a case where

a customer wants to search for a "diving watch," but the item has been saved as "water-resistant watches." Traditional relational databases would not be able to get those particular watches because the keywords are not an exact match; the wording differs. This can be very inconvenient; an item could have been sold, but because of a slight difference in the keyword, it isn't. Or, there is a specific technical question that a customer wants to ask, but just because it wasn't phrased exactly as it is in the database, the customer doesn't get an answer.

To be able to solve this, we have to use vector databases. For the Tanui Watch Store, we will be using **Qdrant**. In this case, instead of just storing data, Qdrant allows us to store data in higher-dimensional mathematical vectors usually called **embeddings**. These are capable of capturing the meaning of the words rather than just the spelling of those words.

According to **Qdrant (2024)** documentation, they stated that these databases are equipped with an advanced algorithm: **Hierarchical Navigable Small World (HNSW)**. This allows the system to perform similarity searches within the shortest time possible. Think of a scenario where a user asks a question like, "Can I wear this watch while I'm swimming?" The system is capable of instantly digging into the technical data and finding information about the ATM ratings and how the crowns are screwed down. Even if the user doesn't use technical jargon, the vector database is able to bridge that gap and find the relevant information from the database based on the user's query.

2.6. Localized Digital Payments in Emerging Markets

Another question that needs to be answered is: do the payment systems in the Global North work in the Global South, or do we have to localize our payment methods? Is it realistic to copy-paste the idea of payment into a region where they are not exposed to card payments or PayPal, to people who are used to mobile banking? And is the vice versa true?

In this thesis, my major focus will be in East Africa, and Kenya to be specific. Most of the adult population in Kenya isn't exposed much to card payments, mostly because of the infrastructure as well as the lack of exposure to this kind of payment method. Since the introduction of M-Pesa in the early 2000s, it has revolutionized payments in Kenya; over 90% of Kenyans do have access to M-Pesa, at least among the adult population. Therefore, copy-pasting the idea of card payments in Kenya is unrealistic and may cause a significant loss of income to companies who solely use card

payments and PayPal. When thinking about how to approach this, I remember a time when I wanted to make a payment while I was back at home in Kenya, and the moment I saw "pay by card" or by "PayPal," I stopped there and I didn't continue with the order placement. This is not just my story; this is the story of millions of other people who are living in the Global South. If the payment method is not localized, then we will have a lot of cart abandonment, especially in the checkout pages of e-commerce websites.

In Kenya, we are used to mobile banking, and this has been made possible by the consumption of APIs provided by Safaricom under their Daraja API intervention. This can be easily integrated and streamlined by the idea of an STK Push or Sim Toolkit Push functionality, which can be easily incorporated into an e-commerce website. Instead, a customer can enter their number and receive a prompt on their phone for them just to enter their PIN to authorize the transaction. By doing this, it makes it easier to pay for orders online, and this will help businesses because they do not have to fear cart abandonment, especially in the checkout.

The integration of M-Pesa in the Blazor store of the Tanui Watch Store requires us to put into consideration how we can ensure that there is a secure asynchronous callback mechanism. This will be discussed later with the idea of a three-way handshake. This will involve a merchant server, a payment gateway, and a user gadget or handset. By utilizing the .NET security features, the HTTP client, and SignalR, the store will be able to maintain a real-time connection, especially when making transactions. This will enable us to inform the customer of a successful confirmation after making a payment, or an error message when a payment is not completed for whatever reason.

2.7. Summary and Synthesis

While there is extensive research on Blazor, RAG, and M-PESA individually, there is a notable gap in the literature regarding their unified application within the luxury technical retail sector. This thesis will try to fill that gap by presenting a cohesive architecture that addresses both technical expertise as well as transactional localized compatibility.

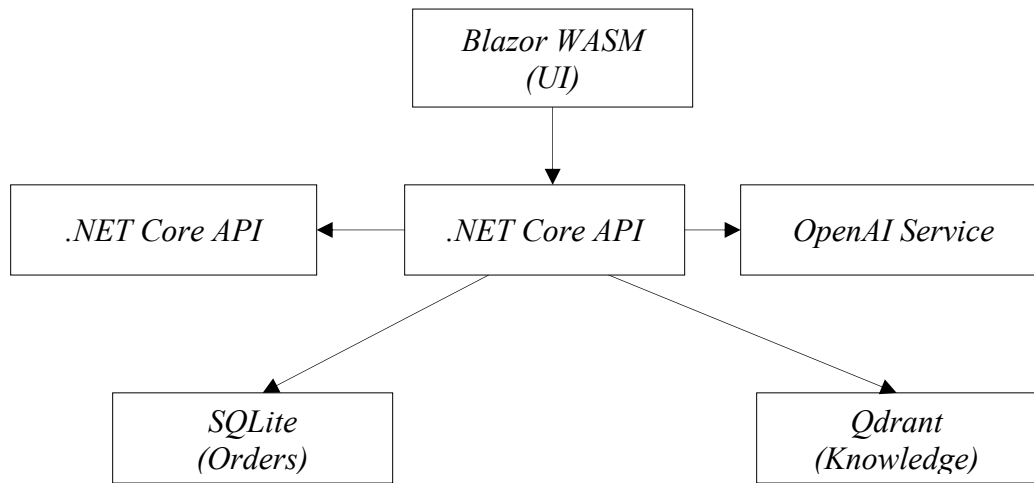


Figure 2.2: A high Level System Architecture Design for the Tanui Watch Store.

CHAPTER 3: SYSTEM DESIGN & ARCHITECTURE

3.1. General System Architecture

In the designing of the Tanui Watch Store, a lot of consideration was given to ensure it meets the required standards of luxury horology. This includes the leveraging of modern distributed system patterns. To develop this store, an N-tier layered architecture was considered to ensure that we separate the three major tiers: the presentation layer, the business logic, and the infrastructure services. This modular approach is very critical, especially if we have to integrate AI agents like OpenAI, as well as the M-PESA gateway, while also putting into consideration the idea of independent scaling and easier testing of the core components.

3.1.1. The N-Tier Layered Pattern

The Tanui Watch Store is divided into four primary tiers.

- The **Presentation Tier** consists of the Blazor WebAssembly client, providing a reactive and near-native desktop experience for the user.
- The **Application Tier** (API) helps in the orchestration of the business workflows, handling DTO transformations and the service coordination.
- The **Domain Tier** contains the core entities and business logic (e.g., pricing calculations, order validation).
- The **Infrastructure Tier** manages external dependencies like SQL persistence, Vector search, and third-party APIs (OpenAI, Daraja).

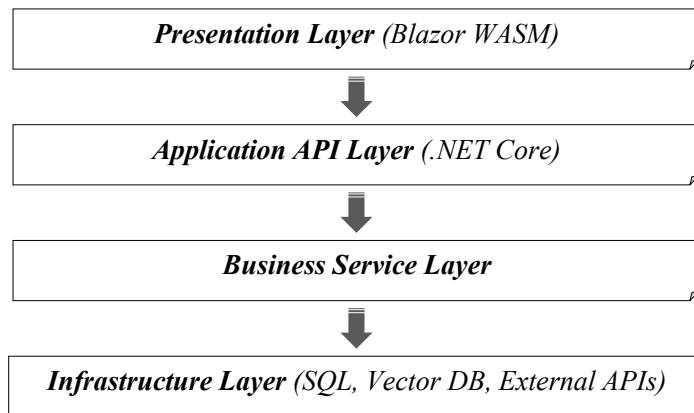


Figure 3.1: Illustration of the high-level layered blueprint of the Tanui Watch Store

3.1.2. The Repository and Unit of Work Patterns

To ensure that we keep the system and make it easy to maintain, something had to be done. The idea of the Repository Pattern had to be put in place. What is this? This hides how data is actually stored or fetched. This is achieved by creating interfaces like *IProductRepository* or *IOrderRepository*, which will help us maintain the business logic without caring as to whether the data comes from a local SQLite file or an external API, especially during testing.

Secondly, the idea of the Unit of Work pattern enables us to manage multiple operations as one complete action. If it works, then everything works; and if it fails, then everything fails altogether. Take, for example, when a customer places an order: the order and its payment must both be saved. In a situation where the payment fails, then there is no need for the order being left hanging and incomplete.

3.2. Database and Persistence Engineering

The biggest challenge I had while designing the Tanui storefront was how data is going to be stored. And as this will be discussed below, the baseline is; the system will be having two different database methods. First, we will have one that will be able to handle structured data, like our orders and payments, to ensure that everything is accurate. The second one will handle our meaning and the context, which would help to manage our chats by understanding human language, processing information, and answering like a human. By doing it this way, our system will be able to manage strict transactions while, at the same time, understanding natural and human-like interaction.

3.2.1. Relational Schema Deep-Dive (SQLite/EF Core)

The first database follows a relational schema where we will be using *SQLite* with *Entity Framework Core (EF Core)*. This is where data will be organized to avoid data duplication and to ensure everything is kept clean. Take, for example, a *Product* table that is connected to a *ProductVariant* table. This allows for different versions of the same watch, including variations like strap material, dial color, or bezel type. The *Order* table sits at the center of everything to ensure smooth relations to every purchase. This structure helps us effectively manage payment transactions, including the integration of *M-PESA* for handling local mobile payments.

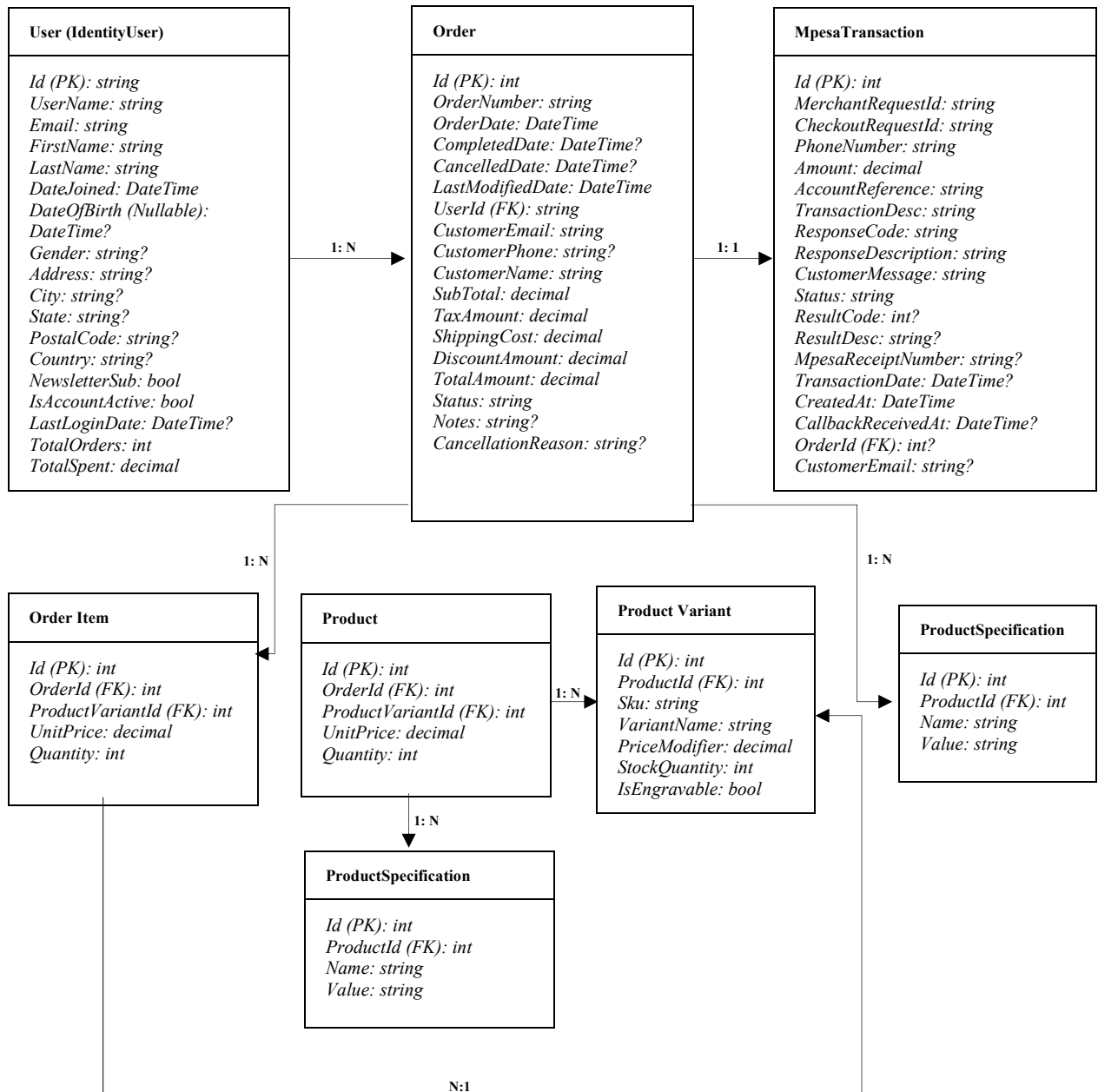


Figure 3.2: A detailed Entity-Relationship Diagram for Tanui Watch Store

3.2.2. Chatbot Persistence Logic

Apart from the relational schema, we have the Chatbot Persistence Logic; what this does is keep track of every conversation and analyze them. The 'ChatConversation' has a table that stores the overall chat session, this includes the summaries that are created by the AI, as well as how many tokens were used to generate the response.

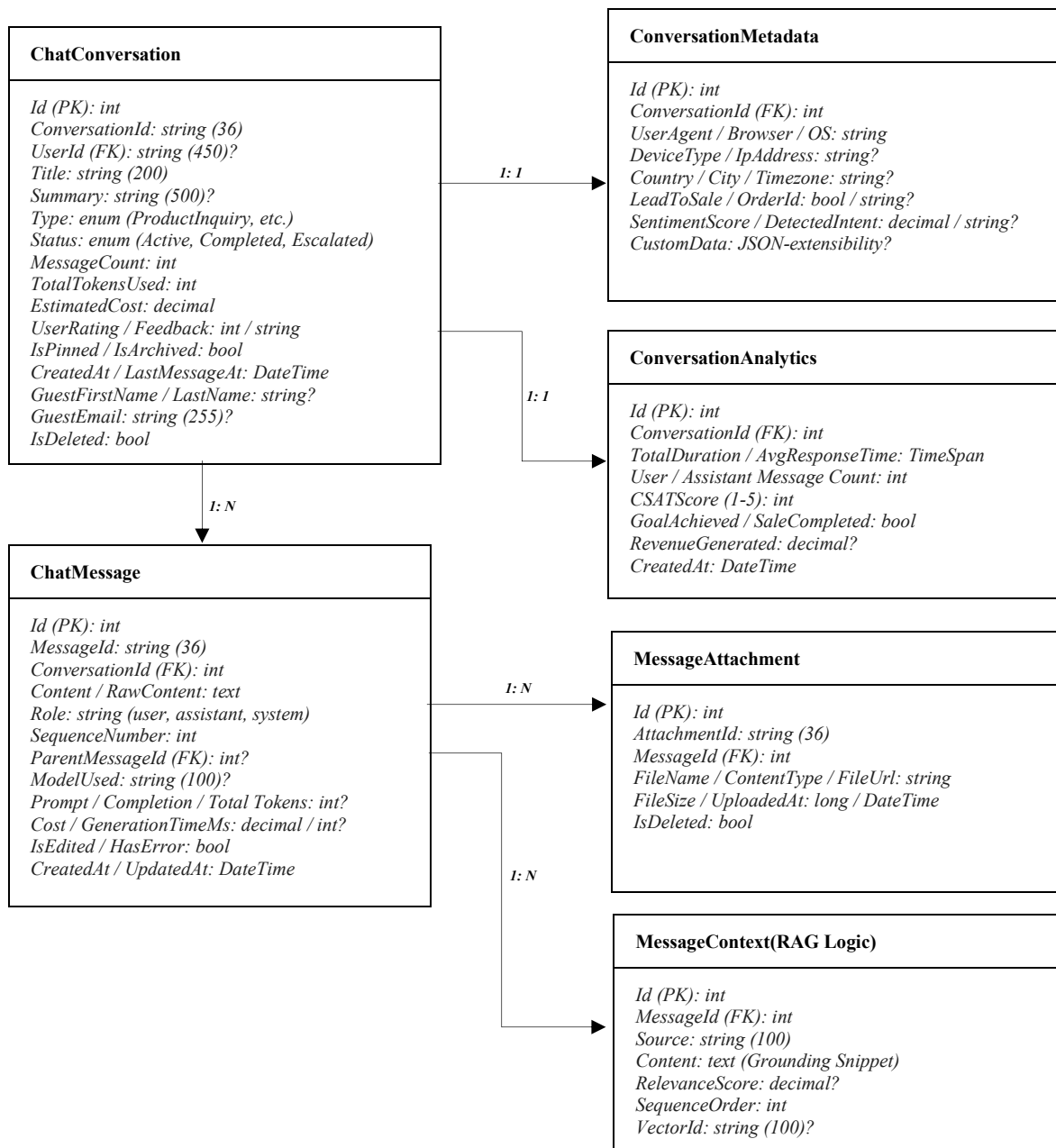


Figure 3.3. An Illustration of the Chatbot's Database Schema

This `ChatConversation` is connected to the `ChatMessage` table, which helps in keeping both the user conversation as well as the AI's response. Also, there is a `MessageContext` table which stores the exact pieces of information that would have been used to generate an answer for every user query.

3.2.3. Non-Relational Vector Storage (Qdrant Topology)

A special database is used called Qdrant, which help with the facilitation of semantic searches. This database works differently from the traditional SQL databases. Regular SQL databases store data in columns and rows, and therefore to search for something exact name or ID is required, and if there are any differences in any of the search and what is in the database, then it assumes that the item or the information is not available in the database.

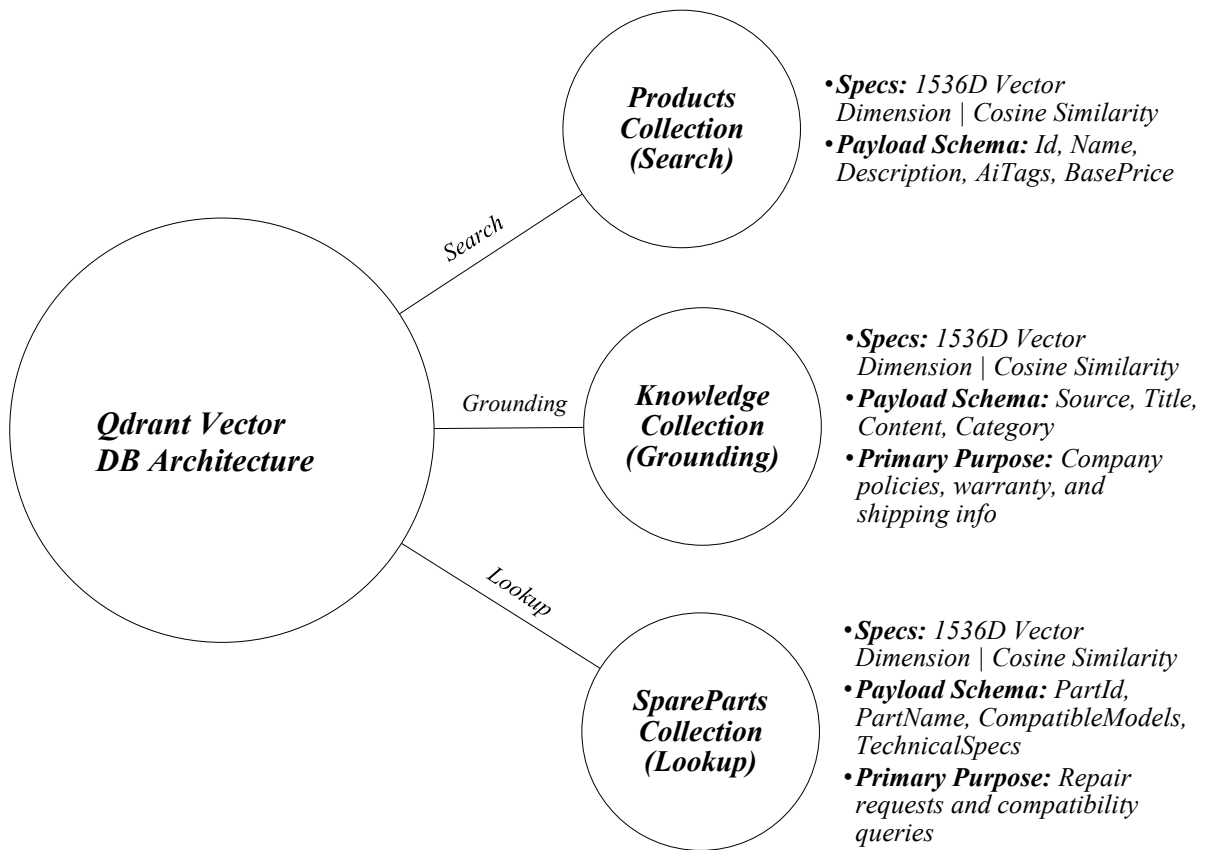


Figure 3.4: Visualizes the vector collection structure in Qdrant

With Qdrant, everything is different, data is not stored in raw values but rather in points in a multidimensional space, the individual points represent the meaning of the data instead of the data itself. In this kind of multidimensional space, the distance between any data point matters, as it carries similar meaning if they are close together and very different meanings if there are far apart. Therefore, what is important is the intent behind every word, not the words themselves.

3.2.4. Multi-Model Data Orchestration

Ensuring that the SQL database and the vector database stay in sync is the trickiest part of the system. The SQL database will store our official product data, like the names, prices, and specifications; while on the other hand, our vector database, Qdrant, will store the meaning-based version of the data so that our AI will be able to understand and search more efficiently and intelligently to help answer our customers more easily.

To make sure that they stay up to date, the system uses a hybrid process. That is, whenever a product is updated in the SQL database, it should automatically trigger a background task. This task should be able to take the updated product information, convert it into a vector format again, and help us to update the matching entry in Qdrant. This process should happen asynchronously, meaning that it should run in the background without slowing the main application down.

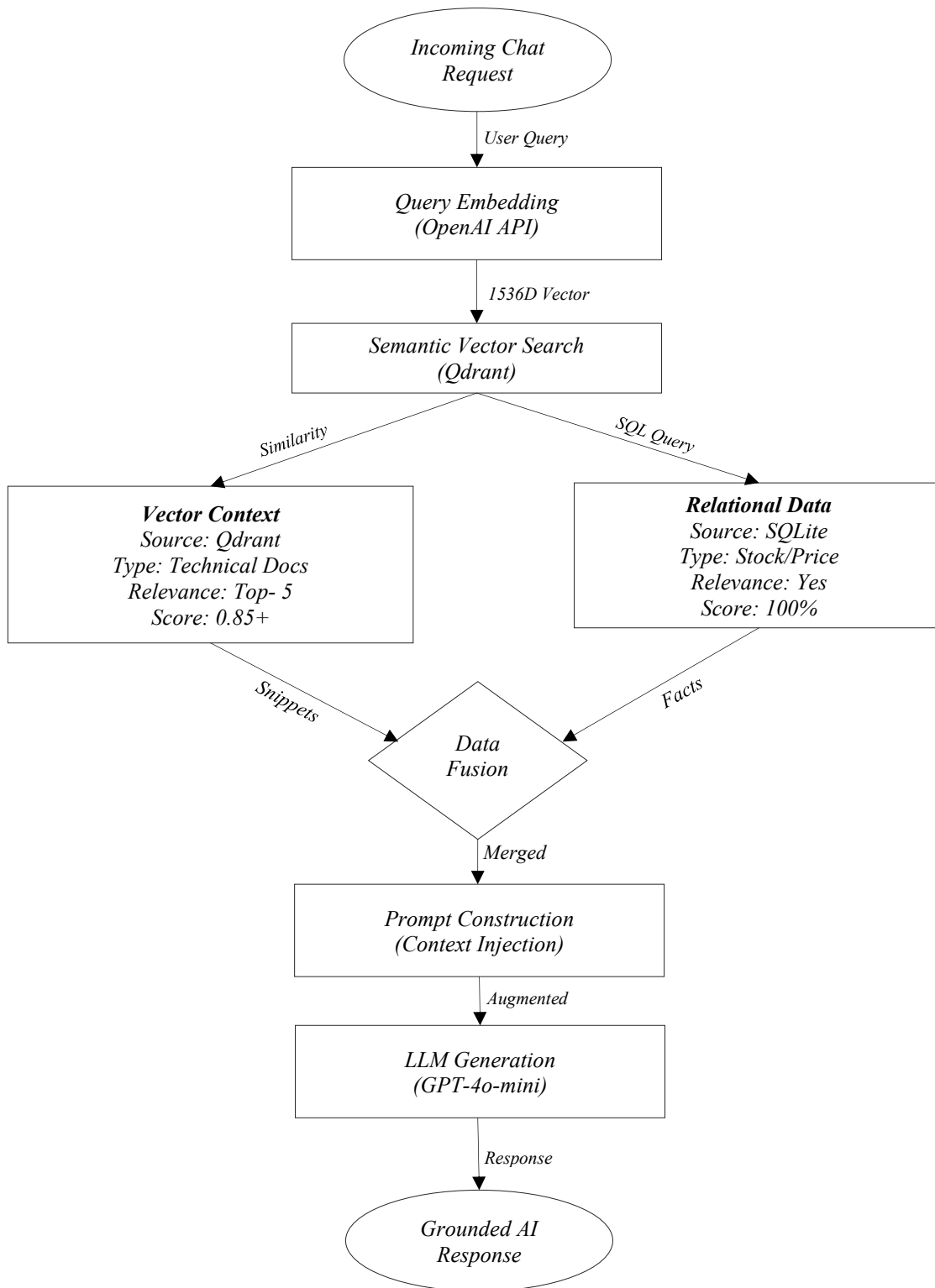


Figure 3.5: Illustration of the joint logic between Relational and Vector stores.

3.3. AI Agent Architectural Framework (RAG Logic)

The AI agent that will be implemented in the Tanui Watch Store stands out because it is built using Retrieval-Augmented Generation (RAG). This makes the AI more reliable; instead of just guessing or answering without fact-checking, the system is able to first look up real information from trusted sources, such as company documents or databases. This reduces the chances of the AI giving incorrect or made-up answers, a phenomenon known as hallucination.

The main job of this subsystem is to ensure that it takes those unclear or casual user requests and turns them into more accurate technical responses. Take, for example, a customer who asks a question that is hard to understand; our AI should be able to get the intent and match it with the correct product details or specific specifications. These specifications should be based on the company data, and this should be able to help us reduce the gap between how people naturally communicate and how we can precisely understand them and answer their queries.

3.3.1. The Embedding Pipeline

What is an embedding pipeline? This is basically the important steps in which the AI system works. Now, it's able to take the user's message that is written in human language or a normal language and be able to turn it into a mathematical format, so our RAG system will be able to understand and compare and give an answer to the query. So, for embedding, instead of working with words directly, they convert those words into vectors. And a vector is essentially a list of numbers that represent the meaning of a word or a sentence. To be able to do this, the Tanui Watch Store will make use of OpenAI's embedding model called `text-embedding-3-small`. This model has the capability of reading entire words or sentences and understanding the context of that sentence; it doesn't just understand the keywords but rather understands the context. It's easy to tell the difference between similar words even if they are used in different situations, and this makes the search much smarter than the normal basic keyword matching as it is in traditional databases like SQL.

We can't just send our text directly to the embedding model. First of all, we have to ensure that it's clean, and this will be handled inside the vector DB service. What happens here is the removal of things that do not add any meaning, I think of things like emojis, HTML tags, or special characters that are not helping us in answering the user's questions. Also, there is the issue of spacing; it

ensures that there is standardized spacing and also converting everything to lowercase. By doing all this, we will ensure that our text will have very little noise and it'll be easier as it will be converted to vectors and later on sent to the vector database, Qdrant, where it makes the finding of similarity of those pieces of information very easy.

In the Qdrant vector DB, the first thing that happens is what we call similarity search, and this is done by a method called Hierarchical Navigable Small World (HNSW). This method will allow our system to find the closest matches to the user's query even in a very large dataset. To be able to do this, this method uses cosine similarity instead of just comparing the exact values. And what is cosine similarity? This is simply how close two vectors are in terms of direction, not distance. Think of it as a way of checking how similar the meanings of words are, and because related ideas are more likely to be clustered together even if they have different wordings.

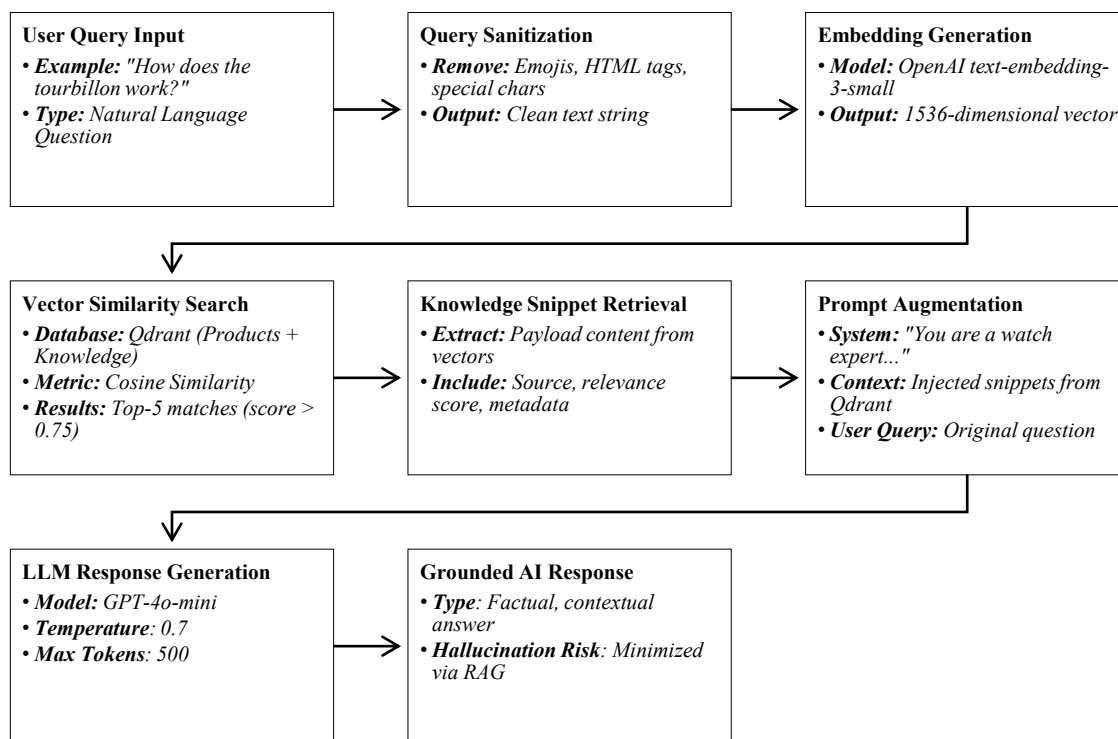


Figure 3.5: Details the internal logic flow of the RAG agent.

Finally, our system will select the top five most relevant results, and this will come alongside their relevance scores. These results are passed to the AI model as context, and this will later help us in generating more accurate and more grounded responses. It usually takes around 200 to 300

milliseconds because our system will need to communicate with OpenAI servers which will then be able to generate the embedding; however, this delay is handled using asynchronous processing. So basically, it means that our chatbot will still feel fast and responsive to the user.

3.4. Payment Integration Architecture (M-PESA/Daraja)

The finalization of payment in the Tanui Watch Store will be handled through the integration of Safaricom Daraja 2.0. This is because mobile payments do not usually happen instantaneously and often have to undergo a complete asynchronous process; so, the system is designed to carefully track each and every one of our customers' payments and store every piece of information reliably. When this occurs, we need to provide real-time updates to our users so that they know what is happening with their payment. This is to help them not leave their carts because of delays, and to ensure that each and every transaction undergoes a payment without having them wait for long times before they get updates on their payment details.

3.4.1. The M-PESA STK Push Handshake

The M-Pesa STK push handshake process starts when a user submits a checkout request on our page. The M-PesaService will create a secure and time-based password using the merchant shortcode and passkey. Then, it will send an STK push request to the payment system, like the user's phone. What the user will be required to do is just enter their M-Pesa PIN; in our system, it will show a pending status instead of just putting us into a wait. This will ensure that the website will stay responsive and avoid freezing while the payment is being processed.

The system will then have to wait for a callback, and to enable this, I created an endpoint called a webhook—this is just /M-Pesa/callback. This can be accessed by using a hosted website; in my case, I used my own hosted website and just added the endpoint, and I was able to get a JSON response. So basically, this endpoint will help us to receive any update from the Safaricom Daraja API once the payment is processed. The moment the user enters their PIN, the payment will be handled by Daraja, who will then send back a JSON response containing the details to show whether the payment was successful through what we call the ResultCode, also, it would send the receipt number, the transaction time, and the amount paid.

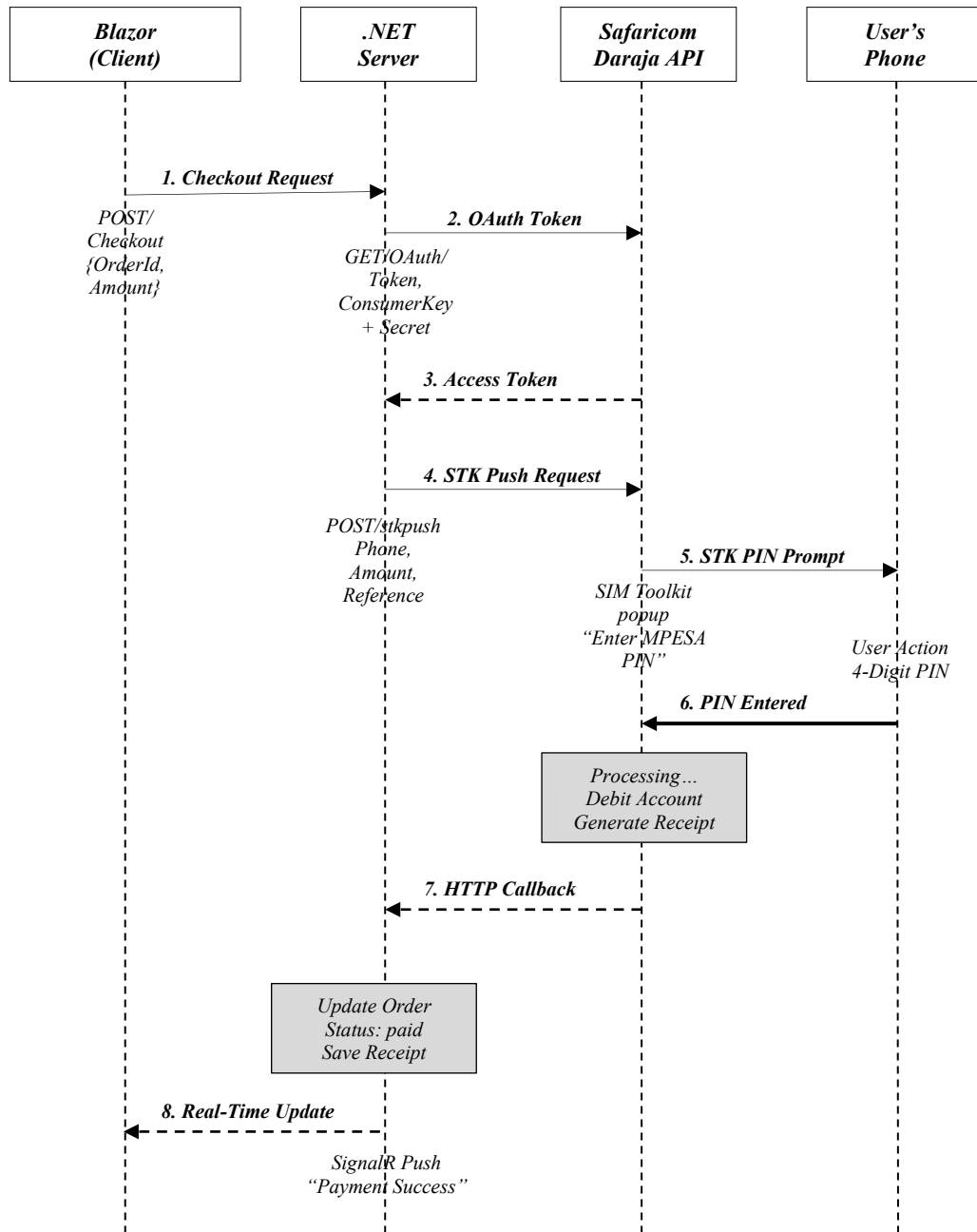


Figure 3.6: An illustration of the asynchronous sequence of an M-PESA transaction.

The system will be able to check if the callback we are receiving is valid, and this is done by verifying if the request matches the CheckoutRequestID with the one in the M-PesaTransaction

table. Once everything is confirmed, the order status will be updated to "Paid" and the receipt number will be saved for record-keeping. To keep the user experience smooth, the system will use SignalR, which basically sends real-time updates to the client. As soon as the payment is confirmed, a status of PaymentSuccess is sent immediately to the client interface. This will help the customer not have to keep refreshing the page to check if things are working efficiently. Lastly, regarding setting a timeout: our system will have a 60-second timeout, and if no callback is received within this time, the transaction will be marked as expired and the user will be asked to try again. This allows the system to avoid staying stuck in "pending" for a long time.

3.5. Security and Identity Framework

And for a luxury platform, one of the most important things is security. We need to protect the customer's financial data as well as our watch information, and to achieve this, we need to ensure that we include a layered security approach. This will rely on JSON Web Tokens (JWT) that will securely manage our user sessions and also control how we access the data on the databases. Also, we needed to use Cross-Origin Resource Sharing (CORS); this will enable us to control which external websites are allowed to communicate with our system. This, together with other measures, will be able to ensure that only those authorized users and trusted resources are able to access our platform, and also to keep our customers' information as well as sensitive business data as safe as possible.

3.5.1. Token-Based Authentication Flow

The authentication system will be handled by the TokenService, and how this works is it creates a secure login token called JWT when a user signs in successfully. This token will be carrying information about the user, like their roles—if they are an admin or basically a customer—and they are required whenever the user will need to perform any action that will change their data. By storing these tokens in a secure way, like HTTP-only cookies or using HTTPS, the system should be able to protect against some of the most common attacks such as XSS or CSRF. The JSON Web Token needs to follow a standard structure that will include three parts: a header, a payload, as well as a signature. The header will basically define how the token is signed. The payload will contain the user details like the user ID, the email, the role, and when the token expires or the expiry date of the token. Lastly, the signature is created using a secret that will be stored safely on the server;

this will ensure that the token cannot be tampered with, and this would ensure that if anything in the token is changed, then the system will easily detect it during verification.

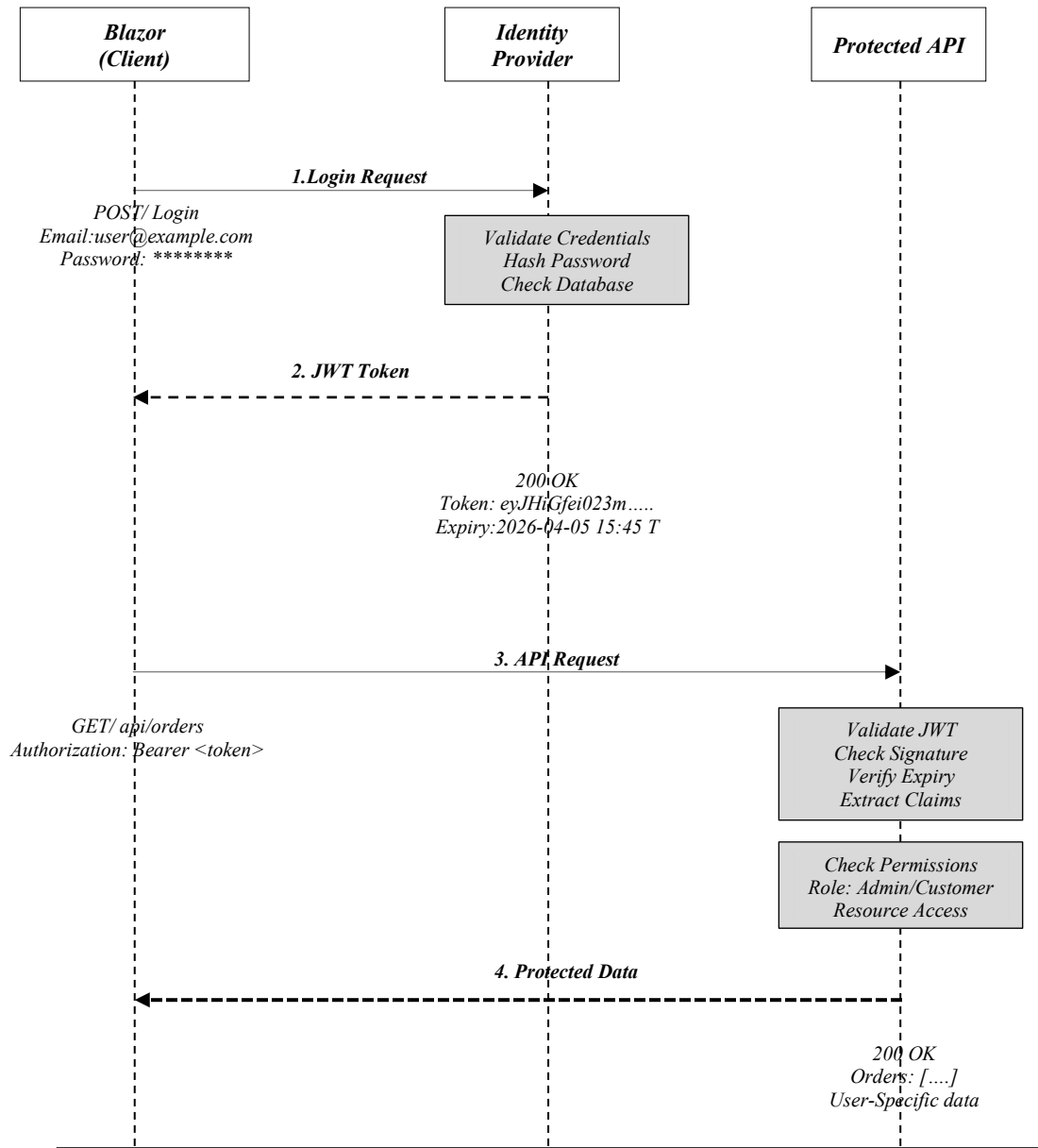


Figure 3.7: The details of Tanui Watch Store secure claim-based authentication workflow

To be able to improve security and usability, those tokens need to expire only after 24 hours. So, we need to have a refresh token system so that we allow our users to stay logged in without the

need to have to enter their credentials again. This will help us to avoid keeping tokens active for too long, which for this case will be very risky if they are stolen. And on the API side, we need to ensure that every request needs to go through what we call an authentication middleware. What this middleware will do is it will check the Authorization header and ensure that it contains the token in the format Bearer <token>. This will be needed to verify if the token signature has expired or not, and also it will help us extract the user's information. For example, if the token is invalid or it has expired or it's missing, then the system should be able to return a 401 or unauthorized response, and this should be able to direct the user to the login. And for sensitive routes like /admin/products, then our system should be able to check user roles; and this will help us to ensure that only those users with the correct permission, like the admin, are the only ones who are allowed to access those protected endpoints.

And to be able to improve security even further, the Tanui Watch system will use Cross-Origin Resource Sharing (CORS). This system will help us to limit which websites can access the API, and only those trusted domains will be allowed to send requests; this will help us prevent other websites from misusing users' login sessions.

That means all communication with the API will have to be protected using HTTPS with TLS 1.3. This will help us to encrypt the data while it is being sent and ensure that it's kept safe from attackers who might want to try to intercept and read all valuable information.

3.6. Real-time Infrastructure and SignalR

One major problem is delays. This may be between actions and responses or any other kind of delay. So, to be able to deal with this, the Tanui Watch system will be uses **SignalR**. What this does is basically to send updates instantly. And this will be very important, especially for payment updates or tracking orders in real time.

When a browser keeps asking a question like "is my payment completed?" to the server repeatedly; it causes what we call polling. This wastes a lot of resources, and this in the long run makes the system to slow down.

To be able to fix this, we have to make our system more efficient. And what is the best approach? It's to create a constant connection where the user and the server will have a dedicated connection

by the use of **WebSocket** technology. And that means that instead of a client asking again and again, the server keeps on sending updates immediately when a change happens.

Basically, the SignalR setup is working like a central messaging system where the server will have the main hub that sends messages. Take for example, when the system processes a payment through an M-PESA callback, it can send a message like a payment status to the connected client. This message can be sent to everyone or only to a specific user depending the situation. On the client side, the application will listen for these particular messages, and when the message is received then it is automatically updated in the user interface and this will work without the need for the user to keep refreshing the page enabling a smooth, app-like experience in the client's browser.

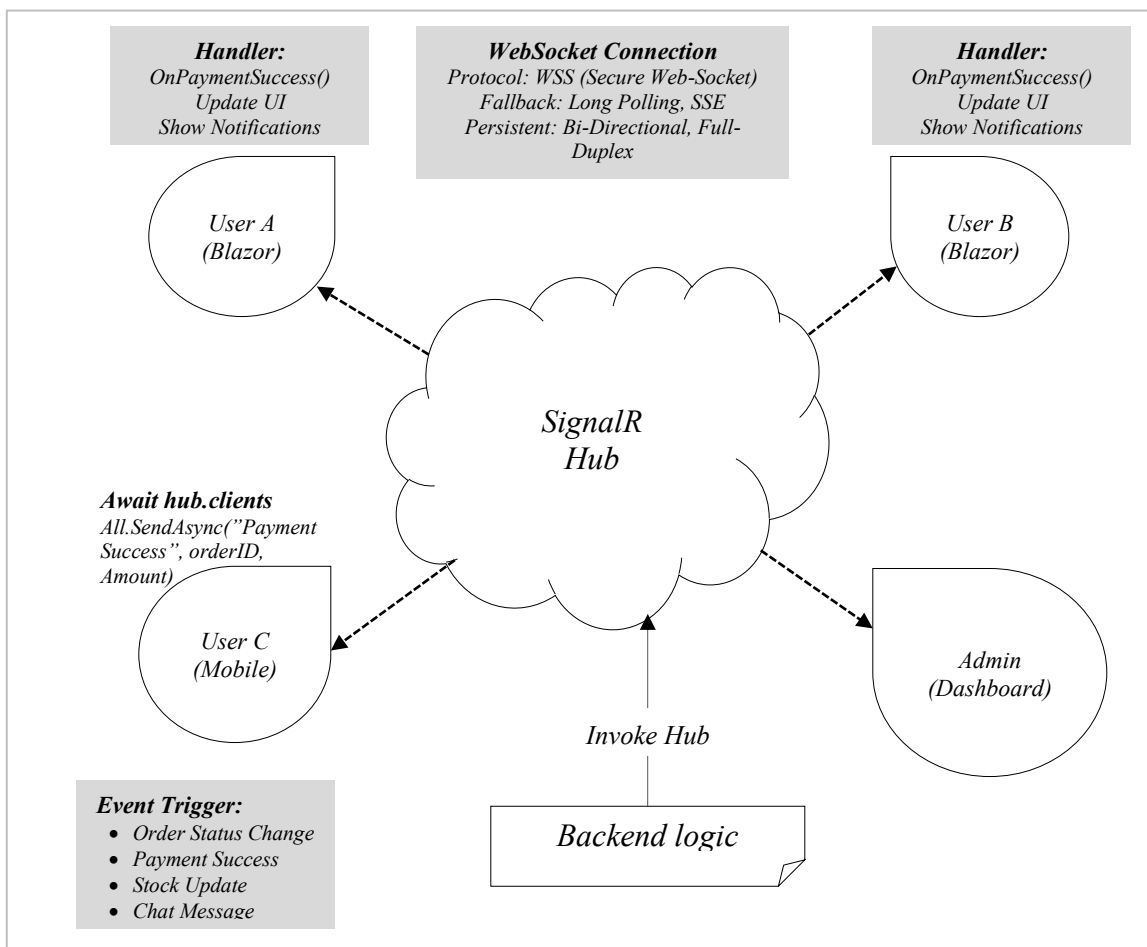


Figure 3.8: Visualizing the real-time event propagation between backend and frontend

SignalR uses WebSockets as its main method because it basically allows a faster and two-way communication. In a scenario where a WebSocket is not available, the system will switch to using other methods like Server-Sent Events (SSE) or long polling. This basically will ensure that our system would work even in restricted networks or those older browsers. The Tanui Watch Store will have to use this to instantly confirm the payments and also help us in notifying the users when the orders are shipped at the same time providing a real-time support, and all of these will help create a fast, responsive, as well as high-quality experience.

3.7. Deployment Strategy and Cloud Infrastructure

The Tanui Watch Store has been built to work on any cloud platform, but we have to choose one platform, and for this, the best option is the Microsoft Azure ecosystem. This will help us to keep everything consistent, like the APIs and our vector services. This will be made possible by the use of packages in containers, which also helps us to ensure that the application will behave the same way in the development stage, the testing stage, as well as in the production stage.

How will the deployment be organized? It will be done in three layers:

1. **The Front End:** This will be built using Blazor WebAssembly and will be hosted on Azure Static Web Apps. I will use a global Content Delivery Network (CDN) to be able to load pages very quickly, often in as little as 100 milliseconds.
2. **The Back End:** The API will run on Azure App Service, and this will use a Linux container on the Standard S1 tier.
3. **The Data Layer:** This will consist of SQLite and the vector database, Qdrant.

The Tanui Watch system will also use a CI/CD pipeline in Azure DevOps, which will help us automatically build, test, and deploy any updates. Updates will be delivered quickly without the need for manual deployment steps. Both the API and Qdrant need to be containerized using Docker, which basically ensures that they run the same way in all environments. This approach will help us reduce any bugs caused by differences between the development, staging, as well as the production setups, making the system more reliable and very easy to maintain.

In regards to the management of data, the system will use **SQLite** with a feature called **Write-Ahead Logging (WAL)** mode. This will help multiple users to be able to read and write at the

same time; this will ensure that it works well for an e-commerce platform where most of the actions happening are reads.

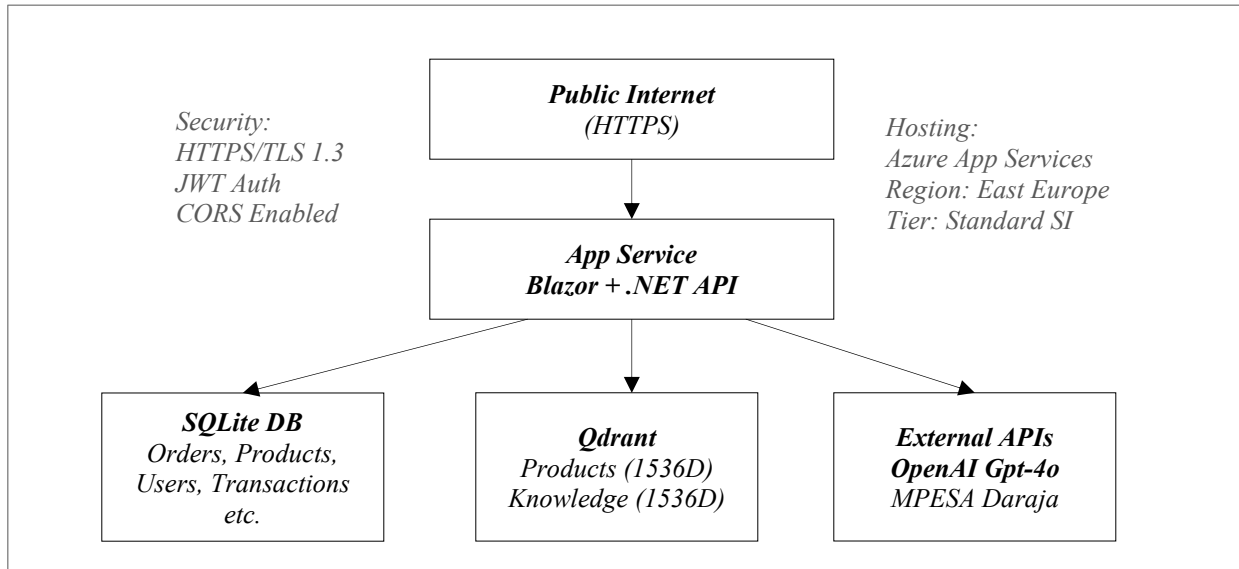


Figure 3.9: Details of the deployment stack of the Tanui Watch Store Ecosystem

To be able to protect the Tanui system, it will create automatic daily backups, and these will be stored in the Azure Blob Storage. This will ensure that the data will be easy to recover if anything goes wrong. Security will be handled in several layers; all communication will use HTTPS with TLS 1.3, and this will keep our data encrypted. Any sensitive information, like the OpenAI or M-Pesa API keys, will be stored securely in the **Azure Key Vault**. The system will also use CORS policies, and this will be able to limit attacks, especially by taking advantage of the benefits from **Azure DDoS Protection** as well as the **Web Application Firewall**, which will help us a lot in guarding against those attacks.

Lastly, how do we monitor and track our performance? The system will use **Azure Application Insights**. What this does is track things like API response times, errors that arise, as well as the system usage; this will help to get a detailed tracing across the services that we offer. There will be constant custom alerts that are set to notify the system if the response time goes above 500 milliseconds or if error rates go over 1%. This will greatly help us to respond quickly when issues occur and ensure the application will run smoothly and efficiently.

Chapter 4: Implementation and Results

According to chapter three presented above, the idea about the architectural blueprint and the design of the Tanui Watch Store system was discussed in detail. So, for this particular chapter, the major focus will be on how that particular design will be realized in the code; by this, I mean the development of the Tanui Watch System.

What are the specific libraries or patterns that were used during construction? What were the practical challenges that were encountered? And apart from that, what were the results that were achieved at the end of the development? Each of these presented here will have references to the relevant design sections that were described in chapter three and their corresponding implementation decisions that led to the required outcome.

4.1. Development Environment and Tools

The Tanui Watch Store was developed using Visual Studio 2022. This was the primary Integrated Development Environment (IDE). It was chosen because it has native support for .NET 8 as well as the built-in Blazor environment, which enabled us to do scaffolding to simplify most of our work. Apart from Visual Studio 2022, the project was also developed using Google Antigravity; this was chosen because of its easy-to-learn interface, and it aided the understanding of the software in detail since it is similar to Visual Studio Code.

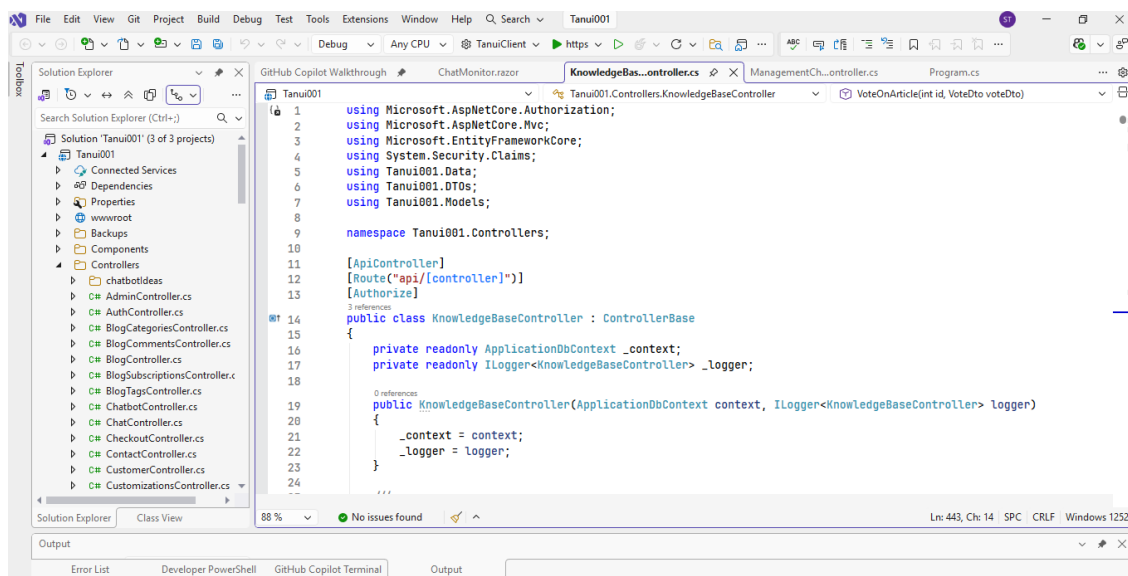


Figure 4.1: Visual Studio 2022 IDE showing the part of the development project involving the Tanui Watch Store

With these and other IDEs, it was much easier to code and develop the system. Also, it allowed us to perform easy integration of Git and GitHub, which enabled easier tracking and version control. .NET 8 gave us the opportunity to use both Blazor WebAssembly for the front end and ASP.NET Core for the API, ensuring a single runtime and the use of shared Data Transfer Objects (DTOs) without any duplication.

Regarding versioning, as explained above, it was managed through a GitHub repository where the use of a feature-branch workflow was of great importance. For each feature or bug fix, an isolated branch was developed and later merged via a pull request. This kept the main branch stable and deployable at any given time. Also, the use of NuGet helped in package management, aiding in getting key dependencies and their roles as shown down below:

Package	Version	Purpose
Entity Framework Core	8.0	ORM for SQLite models, migrations, LINQ queries
Microsoft.SemanticKernel	1.25.0	AI orchestration function calling, prompt management
Qdrant.Client	latest	Connects to the Qdrant vector database
OpenAI SDK	latest	Embedding generation and chat-completion calls
Microsoft.AspNetCore.SignalR	8.0	Real-time push notifications (payments, orders)
FluentValidation	latest	Model validation in form components

Table 1: NuGet Packages used in the development Process

4.2. Infrastructure and Deployment Setup

In regards to the deployment strategy as explained in Section 3.7, this infrastructure had to be configured to support local development and production hosting. Talking about the SQLite database, it was created automatically through Entity Framework Core; this was enabled by its first migration. To be able to run that first migration, it was required to run the `dotnet ef database update` command to apply all pending migrations, which helped in seeding the initial product catalogue as well as the knowledge base records. This is what I can call the zero-configuration approach, which helped in the elimination of the need to create a separate database server during development, ensuring that the entire application would run on a single machine.

Next is Qdrant; it was run as a Docker container using the command `docker run -p 6333:6333 qdrant/qdrant`. During its first run, the vector DB service initialized the required collections with cosine similarity as the distance metric and, as discussed earlier, a vector size of 1536 dimensions. According to the product records, they were embedded so that we can save on time; this was made possible by a one-time seeding operation, and subsequent product additions triggered an immediate asynchronous re-indexing. We have to do this to keep the vectors synchronized, especially with the relational database as it was described in the multi-model orchestration of the design.

Lastly, the API keys for OpenAI and the Safaricom Daraja that were used in this project were stored as user secrets. This was done by using the `dotnet user-secrets set` command and as environment variables, especially during the production hosting environment; this ensures that they are never committed to the Git repository.

4.3. Core Feature Implementation

The next sections and their subsections describe how each core feature defined in the architecture was built. This involves the implementation-specific details, such as the *C#* classes that were used, key coding decisions, and problems that were encountered.

4.3.1. Product Catalog, Cart, and Checkout

Regarding the product catalog, I had to serve them using a `ProductController`, this controller's task was to delegate endpoints, and this was managed by `ProductService`. The `ProductService` then used the repository pattern (`IProductRepository` that is backed by an EF Core Functionality). This really helped to get a well filtered results.

The next question is: how is the shopping cart stored in the browser while a customer is making purchases? To do this, I made use of the local storage, and this was made possible by a service called `CartService`, what this does is to keep the cart saved even if the user moves between pages or faces a connection issue. So, that means that whenever the cart changes, the `CartService` sends an update to the cart icon, which is shown by a badge in the navigation bar and this helps with instant updates.

In the checkout process, this is made possible by a step-by-step Blazor component which includes three stages:

- i. Reviewing the cart.
- ii. Entering the shipping details.
- iii. Making the payment.

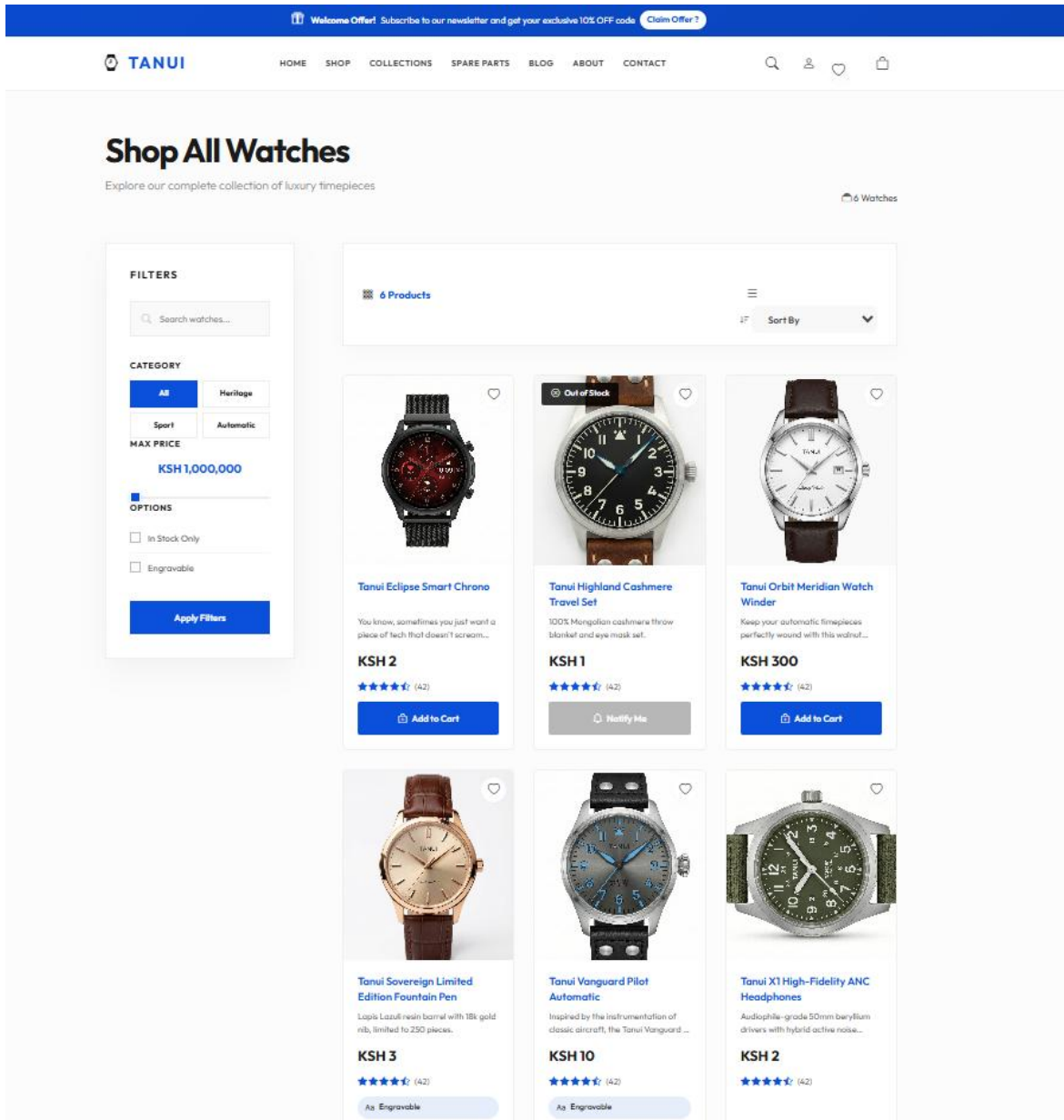


Figure 4.2: Product catalog page, responsive grid with luxury watches, their prices in KSH, and Add to Cart buttons

Each and every step in this process has a validation check; this ensures that everything is filled in correctly before moving forward. This really helps to stop incomplete orders from being submitted. Lastly, regarding the shipping address: it's saved with the order, and this can be seen right away on the admin order management page.

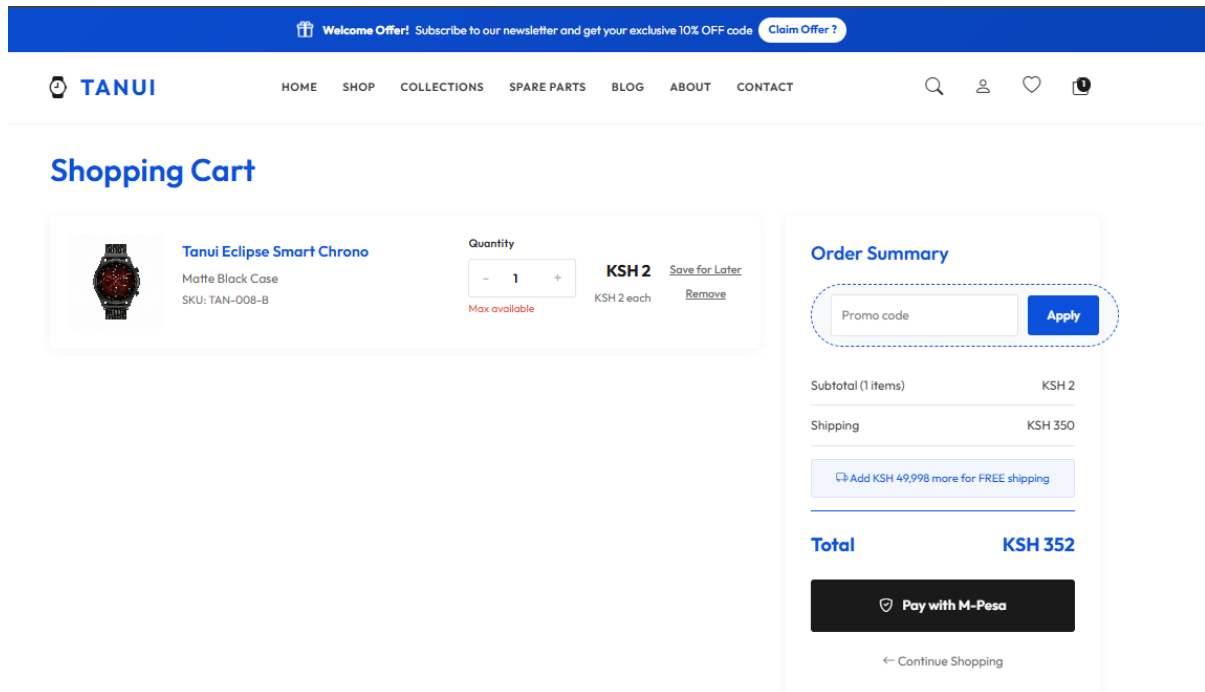


Figure 4.3: Shopping cart sidebar with item list, quantity controls, subtotal, and Proceed to Checkout button

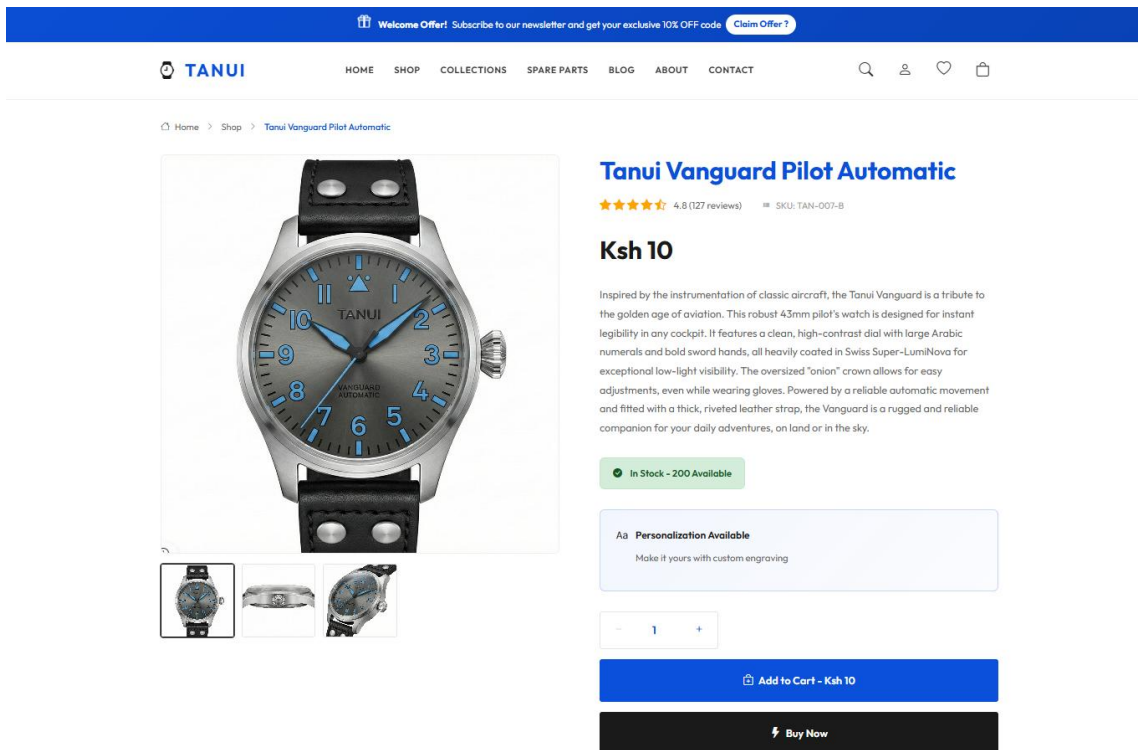


Figure 4.4: Single Product Page, Explaining more about the product image, description, cost, etc

Welcome Offer! Subscribe to our newsletter and get your exclusive 10% OFF code [Claim Offer?](#)

TANUI HOME SHOP COLLECTIONS SPARE PARTS BLOG ABOUT CONTACT

1 Contact 2 Shipping 3 Payment 4 Review

Shipping Address

Full Name *

Address *

City * County *

Ha

Shipping Method

☒ **Pick up from Store**
Ready in 1-2 hours **FREE**

☐ **Standard Delivery**
1-2 business days **KSH 350**

☐ **Priority Courier**
Same-day in Nairobi **KSH 1,000**

Order Summary

Tanui Eclipse Smart Chrono x 1	KSH 2
Subtotal	KSH 2
Shipping	FREE
Total	KSH 2

Figure 4.5: Checkout wizard and Shipping details form

4.3.2. RAG Chatbot Implementation Notes

While developing the RAG chatbot, its steps had to be implemented, and those steps were explained in detail in the design chapter. The system is able to handle two main classes: the VectorDBService, which takes care of the creation of embeddings and works with the vector database, Qdrant; and the ChatbotService, which manages the overall process. This includes things like the building of prompts and making calls to OpenAI.

VectorDbService.SanitizeText() uses three compiled regular expressions to strip emoji code-points, HTML tags, and repeated whitespace before an embedding request is made. This prevents embedding-model artifacts caused by non-textual characters and also doubles as an XSS defence by ensuring raw user input is never forwarded to the LLM without sanitization. The sanitized string populates ChatMessage.Content while the original is kept in ChatMessage.RawContent for audit purposes.

To be able to carefully control how much information is sent to the AI, there is a method and a service that is required. The first is BuildPrompt() and the ChatbotService. A limit had to be set at 16,000 tokens, which basically meant it was the maximum size for any input. So, the system had the ability to be able to count everything that goes into the prompt. This includes the system message and any other useful information retrieved from the database, as well as the recent conversation. If the total becomes too large, the oldest parts of the conversation had to be removed first.

By doing it this way, we made sure that the most important information—especially those facts from the database, are always kept, so even in long chats, the AI still gets the key details it needs to give accurate answers.

To add to that, there is a feature called Microsoft Semantic Kernel (which helped in handling actions automatically). Basically, it defines the eight functions which included searching products, tracking orders, creating support cases, comparing products, and handling admin analytics. Each function is clearly labeled with a name and a simple description, and because of this, the Tanui Watch system can easily understand what each function does and is able to decide on its own which

one to use. Take, for example, if a user sends a message with an order number; the system will be able to automatically call the order tracking function without the need for extra instructions.

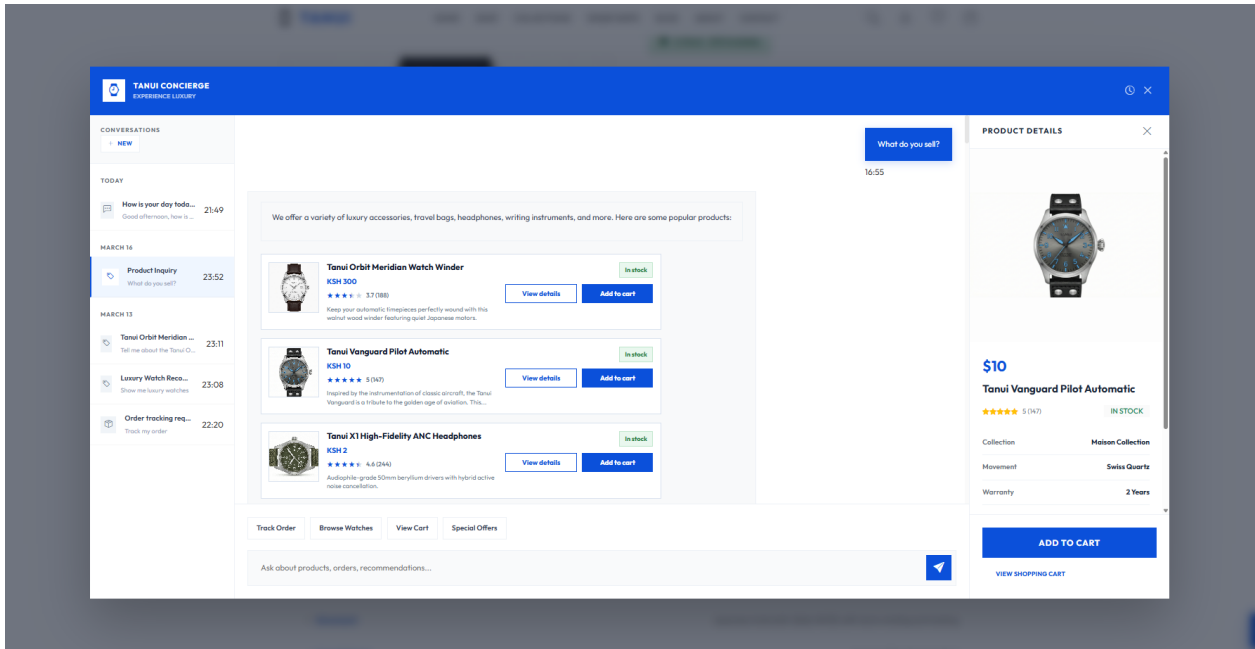


Figure 4.3.2: Example of a conversational made on the Tanui Watch Client chatbot

4.3.3. M-PESA Integration Implementation Notes

As discussed earlier in Chapter Three, the M-Pesa payment process follows 10 steps. In the system development, everything is handled by two parts: the MpesaService, which is responsible for the main logic, and the MpesaController, which basically handles all requests regarding M-Pesa.

One of the important parts is how these phone numbers are handled, because users tend to enter numbers in different formats like 712345678, 0712345678, 254712345678, or +254712345678. With these different types of phone formats, we have to standardize them to avoid errors. To do this, we ensured that we have one standard format for the project; 254xxxxxxxxxx was chosen to ensure we have one main phone format. Just to make it clear, there were a lot of errors during the development of this, and by creating that standardized mobile format, a lot of those errors were fixed immediately.

Next is how to manage the authentication using a token. Basically, the token is saved in memory for 55 minutes; even though it lasts 60 minutes, the 5-minute buffer helps in avoiding it expiring

during a transaction. For each payment request, the system will create a password by combining the business shortcode, the passkey, and the current time, and then encoding it; this is required for sending the payment request.

During development, the system used a Cloudflare Tunnel, which ensured the callback URL was accessible from the internet. This allowed Safaricom to send payment responses back to the local machine without the need to deploy the application online. When a payment response is received (callback), the system checks the transaction ID to make sure that it matches a valid pending payment before it is updated in the database. This prevents incorrect updates. Each payment is given 60 seconds to complete; if no response is received during this particular time, the system marks it as expired and asks the user to try again. This prevents any payment from staying stuck in the pending state forever.

The screenshot shows the checkout process on the Tanui website. At the top, a blue banner contains a 'Welcome Offer!' with a 10% off code and a 'Claim Offer?' button. Below this is a navigation bar with the Tanui logo and links to Home, Shop, Collections, Spare Parts, Blog, About, and Contact. A search icon, user profile icon, heart icon, and shopping cart icon are also present. The main content area features a progress bar with four steps: 1. Contact, 2. Shipping, 3. Payment (current step), and 4. Review. The 'Payment - M-Pesa' section includes an M-PESA logo and a message: 'You will receive an STK push on your phone. Enter your M-Pesa PIN to complete payment.' Below this is a form for the M-Pesa Phone Number, with a dropdown for the country code (+254) and a text input field containing 742178644. A 'Total:' label is followed by 'KSH 10'. At the bottom of the form are two buttons: 'Back' and 'Review Order'. To the right of the payment form is an 'Order Summary' box showing the items: 'Tanui Vanguard Pilot Automatic x 1' for 'KSH 10', a 'Subtotal' of 'KSH 10', 'Shipping' as 'FREE', and a 'Total' of 'KSH 10'.

Figure 4.8: Checkout page, M-PESA payment step showing phone number input and 'Pay Now' button

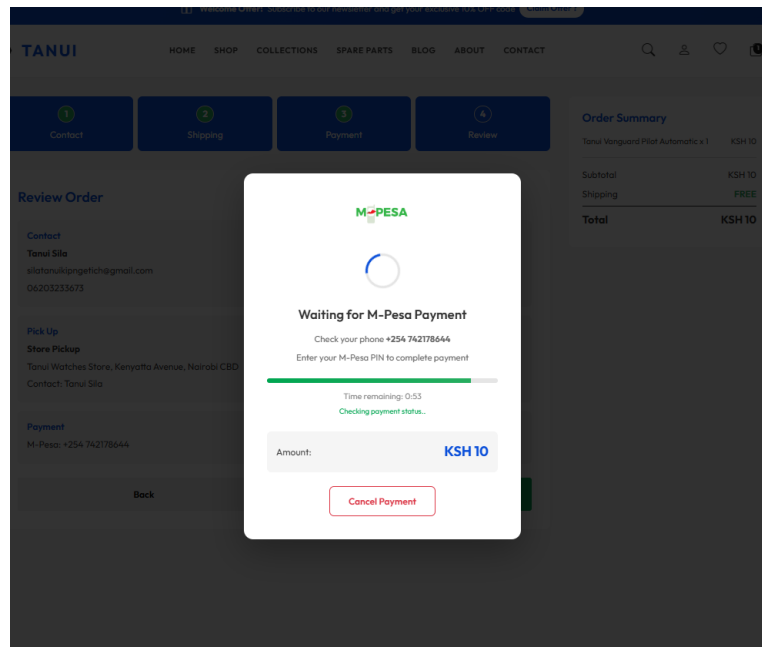


Figure 4.9: Payment pending screen with countdown timer and 'Check your phone for the M-PESA prompt' instruction

4.3.4. Admin Dashboard

To ensure that the dashboard only to allow admin users, a special Blazor layout was used in the development of this project. Any unauthorized access is costly, and therefore, the system has to perform special checks to know users based on their roles, and this is made possible by the use of Json Web Token (JWT); if a user is not an admin, then they are automatically stuck in the login page.

In the admin page, we have to look at the Product management Page, which displays information about the products that we are selling as Tanui Watch Store. For this case images of Watches are clearly displayed in table, with options to filter (Made possible by the use of data tables), other options which includes editing, updating, and deletion of those products. There is also a button that allows the admin to add new products, and just to mention one of the difficult tasks on this particular stage is how images are uploaded into the system, first I have to mention that we have the capability to drag and drop images during the creation of the products. And the images are not just regular images, I had to make sure that I customize the images to have the name of the company, this was made possible by the use of modern AI technology of image generation called

Nano Banana 2 in Gemini. Therefore, after the image has been uploaded, the details are sent to the server, and it triggers a link to be generated and with this link it is then stored with the rest of the product information. And then how does the vector database know about this addition or product update? This is made possible by the AI database consuming the information provided by the generated link, which ensures that update is done in the background, and this will then make the chatbot have up-to-date information about the latest product information.

Order Management page is also one of the most important pages in the admin page, the orders are stored in tabular form, and this gives the admin an opportunity to filter the orders based on their statuses, like “Paid”, “Processing”, “Shipped” etc. Orders can also be filtered based on the day the order was made. One thing to note is the instantaneous updates that happen whenever the admin makes changes in the admin page, this change appears immediately in the Customer’s browser, this is made possible with the SignalR technology. Which makes updates to take effect in real time.

The Analytics page displays the useful business data in a nice and appealing way. Necessary calculations of important information are done, this information include Total revenue, the Number of Orders Made, and even the average order values.

Payment Page keeps track of all the payment information that the Tanui Watch Store is dealing with. Any transaction that is happening should fall into the following categories: “Pending”, “Completed” or “Failed” apart from the status is the information about the amount that it is being processed.

Another important page is the Chatbot Viewer, this page basically gives the admin a chance to go through all the conversations that is made by the users, and the responses that the chatbot is making. This will help when reviews are to be made to understand the performance of the system, and if it is really capable of solving the customer’s queries.

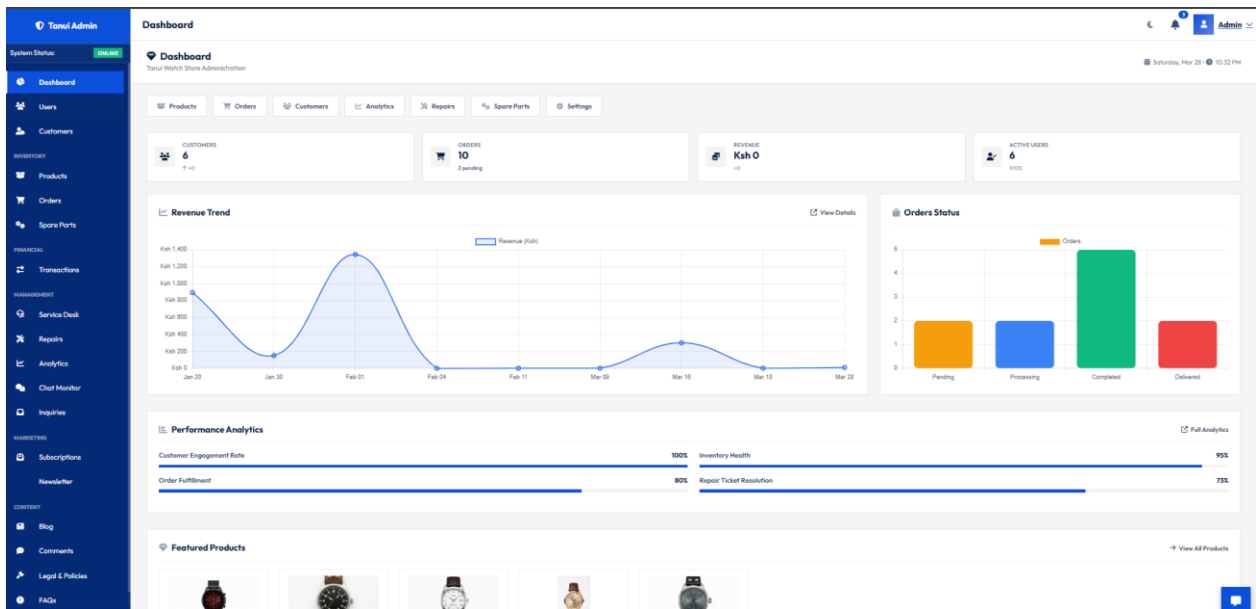


Figure 4.11: Admin dashboard home summary cards for total orders, total revenue, active products, and pending M-PESA transactions

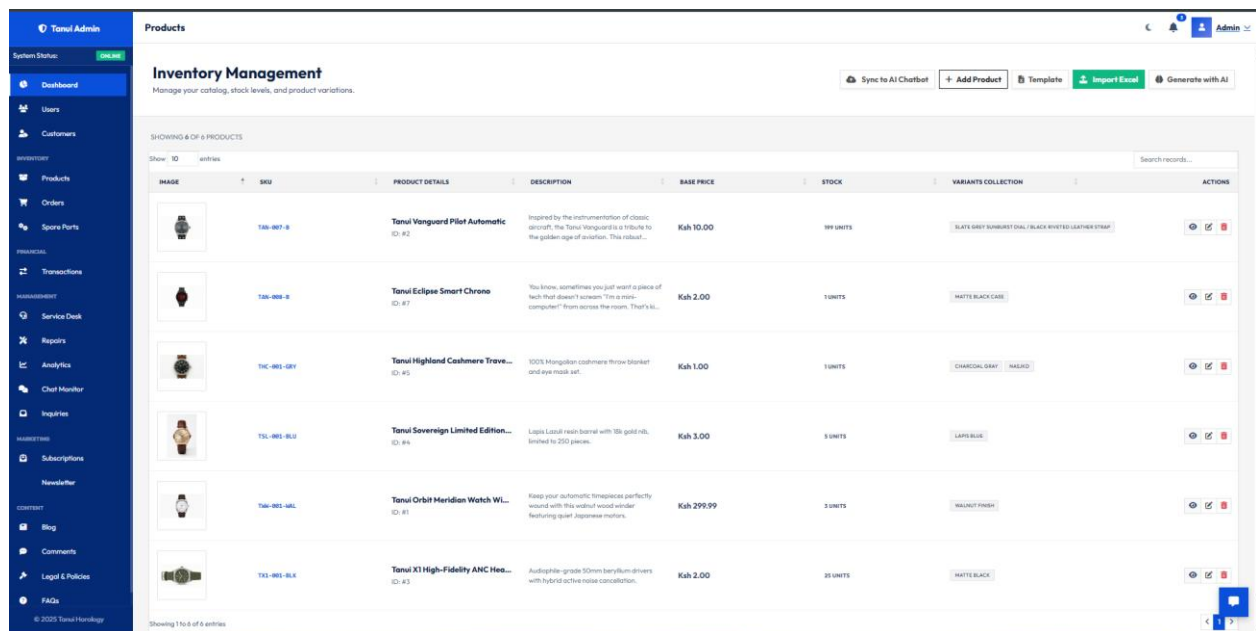


Figure 4.12: Product management page, searchable product table with Edit/Delete actions and Add New Product button

Tamil Admin

System Status: Online

Dashboard

Users

Customers

Inventory

Products

Orders

Spare Parts

Financial

Transactions

Service Desk

Reports

Analytics

Chat Monitor

Inquiries

Subscriptions

Newsletter

Content

Blog

Comments

Legal & Policies

FAQs

© 2025 Tamil Hospitality

Orders

Order Management

Monitor, track, and manage customer orders

10 Total Orders

1 Pending

2 Shipped

Ksh 2,709 Revenue

Search & Filters

Orders (10 total)

Show: 20

ORDER #	DATE	CUSTOMER	ITEMS	TOTAL	PAYMENT	STATUS	ACTIONS
ORD-20260328-F3465AFE	Mar 28, 2026 09:08 PM	Tamil Sila silatamilhospital@gmail.com 📞 +254742178644	1 ITEMS (1 SKU)	Ksh 10.00	PAID	CONFIRMED	
ORD-20260318-177C3F5D	Mar 18, 2026 05:41 PM	Tamil Sila silatamilhospital@gmail.com 📞 +254742178644	1 ITEMS (1 SKU)	Ksh 2.00	PAID	SHIPPED	
ORD-20260316-51FACV72	Mar 16, 2026 03:37 PM	Sila Tamil silatamilhospital@gmail.com 📞 +254742178644	1 ITEMS (1 SKU)	Ksh 2.00	PAID	CONFIRMED	
ORD-20260316-5FA3SC01	Mar 16, 2026 02:13 PM	Sila Tamil silatamilhospital@gmail.com 📞 +254742178644	1 ITEMS (1 SKU)	Ksh 299.99	PAID	CONFIRMED	
ORD-20260309-20FD550F	Mar 09, 2026 03:58 PM	Sila Tamil silatamilhospital@gmail.com 📞 +254742178644	1 ITEMS (1 SKU)	Ksh 2.00	PAID	CONFIRMED	
ORD-20260211-04873DCE	Feb 11, 2026 04:23 PM	Tamil Sila silatamilhospital@gmail.com 📞 +254742178644	1 ITEMS (1 SKU)	Ksh 1.00	PAID	CONFIRMED	
ORD-20264-00003	Feb 04, 2026 04:55 PM	Michael Chen michael.chen@example.com 📞 +1-555-0303	1 ITEMS (1 SKU)	Ksh 229.99	PENDING	PENDING	

Figure 4.13: Order management page, paginated order table with status filter and status-update controls

Tamil Admin

System Status: Online

Dashboard

Users

Customers

Inventory

Products

Orders

Spare Parts

Financial

Transactions

Service Desk

Reports

Analytics

Chat Monitor

Inquiries

Subscriptions

Newsletter

Content

Blog

Comments

Legal & Policies

FAQs

© 2025 Tamil Hospitality

Transactions

Transactions

All payment transactions in one place

Successful

16

Ksh 1,027

Pending

6

Ksh 235

Failed

5

Ksh 240

Total Collected

Ksh 3,027

27 transactions

Search

Phone, receipt, reference...

Status

All

Provider

All Providers

Date Range

All Time

X Clear

All Transactions

27 RESULTS

DATE	PHONE / CUSTOMER	AMOUNT	PROVIDER	REFERENCE	STATUS	ACTIONS
Mar 28, 2026 9:08 PM	📞 0620 323 3673 Tamil Sila	Ksh 10	M-Pesa	ORD-20260328-F3465AFE	✓ COMPLETED	
Mar 28, 2026 9:07 PM	📞 +254 742 178 644	Ksh 10	M-Pesa	1CTQ8AKDF	✓ COMPLETED	
Mar 28, 2026 9:06 PM	📞 +254 742 178 644	Ksh 10	M-Pesa	TAN20260328220636	✗ FAILED	
Mar 28, 2026 9:06 PM	📞 +254 742 178 644	Ksh 10	M-Pesa	TAN20260328220632	✗ FAILED	
Mar 28, 2026 9:06 PM	📞 +254 742 178 644	Ksh 10	M-Pesa	TAN20260328220623	✗ FAILED	
Mar 28, 2026 9:05 PM	📞 +254 742 178 644	Ksh 10	M-Pesa	TAN20260328220623	✗ FAILED	
Mar 18, 2026 3:55 PM	📞 +254 742 178 644	Ksh 1	M-Pesa	TAN20260318185617	🔄 PENDING	
Mar 18, 2026	📞 +254 742 178 644	Ksh 1	M-Pesa	1CTQ8AKDF	✓ COMPLETED	
Mar 18, 2026	📞 +254 742 178 644	...	...	...	✓ COMPLETED	

Figure 4.14: Transactions page, summary cards and searchable transaction list with receipt numbers

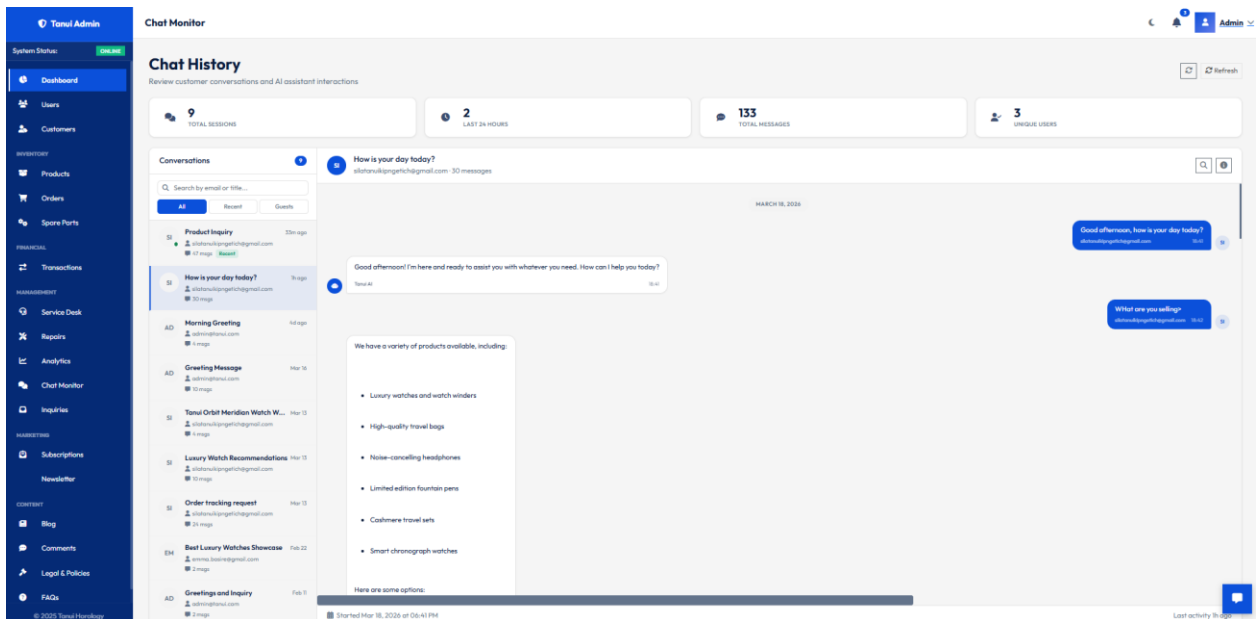


Figure 4.15: Chat History viewer, admin view of a customer conversation with RAG

4.4. Testing and Validation

4.4.1. Functional Testing

The Tanui Watch Store system had to be tested manually, also, I made use of AI to recheck the coding part to ensure that everything is working perfectly, and if there are some better ways in which the system can be structured, the AI testing and recommendations brought a lot of insights that I had not seen before, it helped in the simplification of a lot of complex tasks and processes that I had not thought about before.

Some of the manual test that I did on the system includes the entire browsing experience how the filtering products behaves, how the addition of products to the cart works, be it in the chatbot or the normal single page containing the specific product information, changing quantities, and completing the checkout using the M-PESA test environment.

I also had to test the chatbot using my simple human language and more technical terms that is usually being used in the horological world (watch-related language). Also, I had to see on how products are managed That include how products, orders as well as payments are taken care of in the admin dashboard. this is just to ensure that it performs the task that they are required to perform.

While testing the system, it can't go unnoticed to say that there were some bugs, most of these issues I was able to fix them by myself, but some of them I required some assistance from AI to be able to read my code and find the issue, and help me in finding a solution. I have to just mention a few of those issues:

- **Cart Icon:** I realized that the icon did not update right away after an item was added to the cart. I came to realize that this issue happened because the checkout page was not listening to the changes from the CartService.
- **Chatbot Layout in Small Screens:** The layout was covering the page footer. Fixed by limiting the height of the chat area and allowing scroll-ability.
- **MPESA Payments:** This happened after I noticed that the payment response happened after the system had already marked the initial transaction as expired. This made some successful payments to appear as though they had failed, and to fix this issue, I had to increase the waiting time to 60 seconds, also I added an additional check to verify if the final payment status was correct.

4.4.2. Security Validation

Logins and access controls also needed to be tested to ensure that they worked in a manner that is expected. Trying to access the admin page without logging in will trigger 401 Unauthorized Error, and taking an example of a random user like one of our customers try to access the admin page with their own login details, the system will return a 403 Forbidden Error, this therefore confirms that the only people allowed to access the admin page are the administrators.

Also, MPESA security is crucial, the callback URL had to be tested to ensure it was secure. Any request coming from unknown or unsaved IP address had to be blocked, and only callbacks from trusted sources had to be accepted.

To ensure that our chatbot is secure, I had to perform several test cases, in some cases adding some elements that were not needed like HTML codes of Emojis, and surprisingly, the system was smart enough to clean the input before storing and using it. That gave me the confidence that no unsafe content was being sent to the AI, or sent by the AI to the user.

And finally, is the security of our sensitive information like the Open AI API key as well as the M-PESA secret keys, do we have to take a risk and commit them together with the code to online repositories like GitHub? The answer is a no, therefore, to ensure they are safe, then therefore they had to be put in a safe file that has immunity, especially when all the other files and folders are being committed to GitHub.

4.5. Performance Observations

Performance has to be checked using the browser's developer tools, that is by using the Network panel of any of the available browsers, and with this Network panel is where I got the logs that will give more information about how the server is performing. The results and observations of some of the main features are summarized below:

Operation	Observed Latency	Notes
Product listing API	< 100 ms	Single-user; EF Core query + JSON serialization
Qdrant vector search	40 – 60 ms	Top-5 cosine similarity across ~500 product vectors
Full RAG pipeline	1.5 – 2.5 s	Embedding call + Qdrant + GPT-4o-mini generation
Blazor WASM first load	2 – 3 s	One-time .NET runtime + assembly download
In-app navigation	< 50 ms	Client-side routing after initial load
SignalR payment event	< 1 s	Time from M-PESA callback receipt to UI update

Table 2: Performance Observations using the Network panel In the Developer Tools

Regarding the cost of AI, the development of the Tanui Watch store uses the OpenAI API, and based on the current pricing, it was very cheap. Creating embeddings or using the chatbot model may cost a little bit more, but overall, each conversation is estimated to be around **\$0.05 to \$0.10**. This was low enough to be practical for a small to medium-sized e-commerce business.

4.6. Summary

Just to summarize everything that I did in Chapter 3, First is the development environment, which I chose Visual Studio 2026, together with Google's Antigravity. I used technologies which includes .NET 8 and Docker.

I also employed the use of Blazor WebAssembly for the front End and ASP.NET for the backend. To ensure that the data is stored well in the DB, I used EF Core together with SQLite, and Qdrant as the vector database. Also in Chapter 3, I explained more about the RAG, and how I needed to make sure that the text that was sent and received out of the pipeline was clean, and understandable from both ends of the pipeline. That is the end User and the Qdrant DB. I also need to note that setting the token's limit was crucial.

Next is the idea of MPESA integration. I needed to ensure that I avoid the phone number format headache, more details were explained in the chapter.

The admin Dashboard, needed to be kept safe, so to do that I had to only allow the admins to get logged in to the system, and no any other person who tried to access the admin page was an Unauthorized user.

More information regarding the testing of the system was also discussed in detail. Those security features included login check, access controls and input cleaning.

Chapter 5: Conclusion

The development of the Tanui Web Store is complete, but what is most important is to now combine everything together, and to ensure that starting from the problem Identification, designing of the system all the way to testing was actually done in the correct way, and if it meets the initial plan.

I will try to review the original goals and determine if the goals that that were set initially were successfully achieved, and also point out what may not have been achieved during the entire process.

It also gives some suggestions on what can be done in the future, what new improvements can be added to this system. So basically, the goal is to just get a clearer picture of what the project has achieved and why this project is important.

5.1. Summary of Work

The main idea was to develop the Tanui Watch Store, which was supposed to be a full-stack online shop for luxury watches. The idea was to build it using the Microsoft Blazor Web framework, and its major purpose was to solve two real problems which were noted while I was working in customer support. The first one was how we can easily answer those technical questions in the shortest time possible, and also trying to solve payment issues, especially for those customers who are living in the Global South, like Kenya, by the use of localized payment methods like M-Pesa.

To be able to handle those questions, as explained in the previous chapters, the system used the Retrieval-Augmented Generation (RAG) approach. This is when a user asks a question, the system will first use a Qdrant database to find the most relevant product information related to the query using a 1536-vector space. After getting the relevant information to answer the customer, it sends this information to the GPT-4o mini model, which will then generate an answer; this ensures the AI uses real as well as verified data to avoid guesswork in any situation where the system might just make up information.

To solve the payment challenge, the system that was designed integrated the Safaricom Daraja API 2.0 API. What this does is ensure that customers complete their payment directly by the use of M-Pesa in just one single step without the need for manually entering codes.

Just to talk about the technical side of this project, it was built using ASP.NET Core 8 and made use of Entity Framework Core with SQLite to manage data. Also, the implementation of SignalR enabled the sending of updates in real time, especially in the case of payments. Finally, the system was deployed using Azure.

The client page cannot work alone, so there was a need to include a dashboard, and this gave the admin the ability to manage and have full control over the products being sold, the orders, the transactions, as well as the chat records.

5.2. Research Objectives Revisited

During Section 1.5 of this thesis, there were some specific objectives that were supposed to be met in this particular project, and looking back at each and every one of them, this is how it turned out:

Objective	Outcome	Evidence
Build a Blazor WebAssembly frontend providing a near-native browsing experience.	Achieved	In-app navigation runs under 50 ms after the initial load. The product catalog, cart, checkout wizard, and admin panel all work as designed (Section 4.3.1).
Implement a RAG pipeline using OpenAI embeddings and Qdrant for factual AI responses.	Achieved	The pipeline produces 1,536-dimensional vectors, retrieves the top 5 results by cosine similarity, and injects them into the prompt. Sanitisation prevents hallucination from bad input (Sections 3.3 and 4.3.2).
Build a .NET backend that connects the relational database and the vector store.	Achieved	The hybrid persistence layer keeps SQL and vector data in sync using an async re-embed step whenever a product is updated (Section 3.2.4).
Integrate Safaricom Daraja 2.0 for M-PESA STK Push payments.	Achieved	This was achieved by testing the system using the Darja sandbox, and this was made possible by handling payment callbacks and using a 90-second timeout. Also, it was made possible by normalizing the numbers, as described in Section 3.4 and 4.3.3
To validate that our system is able to reduce query response time compared to human	Partially achieved	The AI pipeline was able to respond in like roughly 1.5 to 2.5 seconds, but no controlled comparison against real human agent response times was conducted. This is acknowledged as a limitation in Section 5.4.

Table 3: Research Objective Revisited

In as much as the four objectives were met, the fifth objective requires an honest explanation. The main agenda of this thesis is to ensure that if something cannot be fully proven, then I should be able to clearly state it without trying to suggest that an objective was fully achieved when it wasn't.

In this particular case, the AI system works fast, but there wasn't any formal comparison with real human customer support agents under the same conditions. Conducting such a study would require a live system with real users, which was beyond the scope of this particular project. Regarding the speed results, they are still very useful as a reference, but for this particular case, they should not be treated as proof that the AI is faster than human agents.

5.3. Research Questions Answered

5.3.1. How can RAG be optimized for factual accuracy in a technical retail context?

To ensure that RAG becomes optimized for factual accuracy, we need to closely monitor it so that it only uses trusted information. To do this, we need to explore four steps:

- Ensure the user's input is clean; this involves things like removing emojis or HTML tags that would interfere with how our system understands the text.
- Ensure the input is converted into a vector and used to search the product database in Qdrant for similar information.
- Ensure the system only keeps the top most relevant results; this is to ensure the results are clear and focused to the point.
- Ensure the results are added directly into the prompt before the AI generates the response; this ensures that the AI is actually answering using real product data.

This approach avoids the situation of the AI guessing or relying on its own memory. While developing this, I realized that this system will need ongoing care. That means the AI is only as accurate as the data stored in Qdrant. In a case where the data becomes updated, the answers given by Qdrant will become outdated; keeping the data up to date is, therefore, an ongoing task and not just something that should be done once.

5.3.2. Does M-PESA integration reduce cart abandonment for Kenyan shoppers?

The next question that is to be answered is: does M-Pesa integration really reduce cart abandonment for Kenyan shoppers? A proper A/B test to measure the actual cart abandonment was not done, so I cannot provide any results in this report.

The implementation of the Tanui Watch Store is meant to show that one of the key causes of abandonment identified was the need for customers to manually dial a USSD code. But since the customers now only have to enter their phone number and get a prompt on their screen—allowing them to simply enter their PIN and receive a confirmation in less than a second through SignalR—it is much shorter than the normal USSD code process.

This results in a shorter wait and fewer chances for mistakes compared to how many Kenyans experience international platforms like credit cards or PayPal. I can say that to truly prove how much this improves abandonment rates, our system should be deployed in a real-world production environment where we can have actual users and track their transactions; this is one of the areas of future work that I recommend future developers put into consideration.

5.3.3. What are the trade-offs of Blazor WebAssembly compared to JavaScript frameworks for AI-heavy applications?

We also need to look at the trade-offs between Blazor WebAssembly compared to JavaScript frameworks for AI-heavy applications. Since the .NET runtime usually has to be downloaded to the browser, it typically takes two to three seconds to ensure it is fully loaded. A common React app will usually appear very fast at the start. However, after the runtime is stored in the browser, moving between pages typically takes less than 50 milliseconds, which is very fast and similar to client-side frameworks like React.

The major advantage of using Blazor in this particular project is shared code. This means I can use the same C# data models and validation rules in both the client and the server. If I were using TypeScript and C#, this code would need to be written and maintained in two separate places, and keeping them in sync manually would lead to errors. Blazor removes these duplications completely.

Regarding the AI part, the choice of framework does not affect the speed; the delay of about 1.5 to 2.5 seconds that comes from the API response time would have also affected a React front end if it had used the same OpenAI API. Therefore, the performance will generally be the same.

5.4. Limitations of the Study

Being clear about what this project did not do is as important as describing what it did. As explained in the introduction of this chapter, one of the limitations was that there was no real user study, and most of the usability feedback came from me as I was testing the application; no outside participants were involved, and therefore no formal usability measures were collected. We can say that the results of a single user cannot be used to generalize for all other users.

The waiting time, or latency, was measured on the development of the system. The system having not been tested by many at the same time for example 10, 100, or thousands of concurrent users would require automated testing. The test that I did in this project were mainly manual, but with little help from AI I was able to perform additional test that I may not have seen, therefore I can say that there were no XUnit or BUnit tests therefore, I can say that there were no automatic safety checks if the code changes.

The M-Pesa integration I used only utilized the Daraja sandbox, so the payments were simulated rather than real, but the whole process of payment was real, as it required real money deduction from my mobile account and it was captured in the system, the only difference is after the payment, the money I used was refunded back to me, but other than that, there are some necessary rules to follow before going live where one would need official approval from Safaricom and would need to follow real financial rules, which were outside the scope of this project.

The system uses SQLite, which many say is not ideal for large-scale production. While it works well during development and small workloads, a larger production environment would need a stronger database like PostgreSQL or SQL Server to handle many users at the same time.

Another limitation is the issue regarding "hallucinations" in the AI responses. While the RAG design was intended to reduce this issue, therefore I cannot say for sure if the RAG was able to remain within the context of what it was supposed to be due to limited sample chats, but it did

perfectly well in answering some test samples, But I would highly suggest the use of tools like RAGAS can be used in future work to evaluate this more accurately.

5.5. Future Work and Recommendations

Using those limitations, I have mentioned above can really help in improving the Tanui Watch System, they are not only useful to extend what has already been built but additionally serve as a necessary step to ensure we have a real user study. If there is a possibility that we can invite Kenyan watch Makers or customers to use this system and then measure factors like the number of tasks completed or how satisfied they were about the system then we could get more information on how to improve the system.

Another thing that needs to be improved is a proper test suite; for example, an **xUnit** test suite can be created to cover parts of the system like the RAG pipeline, the M-PESA callback handling, or how the entire checkout process performs. Without these tests, the code is harder to maintain and make it harder to apply new changes.

To ensure that the system is able to be tested under heavy usage, we need a tool like **k6**, which we can use to simulate many users at the same time this will help me to see how the system will perform under immense pressure. This information will be very reliable in the identification of bottlenecks, especially in SQLite or the API, and based on those results gotten, the system can be upgraded to use **PostgreSQL** or **Azure Container Apps**, which will help in the automation and scaling to handle more users.

It's also important to ensure that we measure the quality of the RAG system over time. We can use tools like **RAGAS**, as explained above. This system will in the regular checks and to determine how accurate or relevant the AI is, making quality something that we can track and improve continuously.

We also can't ignore people who want to use international payment methods like credit cards. To allow these customers to pay in currencies like Euros and Dollars, there is need to make sure these options are available within the Tanui Watch Store website, especially if the plan is for the store to be used by a wider audience beyond the Global South or the East African region to be particular. We have to consider these different modes of payment in our checkout system.

There is also need to create a mobile app, and to do this then **.NET MAUI Blazor Hybrid is at the best choice**. The same Blazor components can be used to create a native app for iOS and Android. This will basically mean that most of the existing logic will be reused, and this in turn will help in saving of cost and time

To conclude, we need to see how we can make the AI more powerful. Right now, the agent mainly answers questions, but it could do things like **Semantic Kernel planning**. This will give it more power to perform tasks like processing returns, checking delivery status, or marking products as out of stock.

5.6. Final Reflection

At the beginning of this thesis, I explained how I worked as a customer care agent in an e-commerce company called Fossil. While I was working in this company, there was a lot of experience that I gained that helped me to think about developing a project that would be able to answer questions that were repetitive in nature and that required a lot of digging into the company documents. Another reason that made me think about this is how sometimes I left customers waiting for quite a long time as I tried to figure out where exactly in the company documents a particular piece of information is, or waiting for quite long times before team leaders could respond to my queries.

My idea was to build the Tanui Watch Store and to integrate the RAG system that will be able to work correctly and to be able to answer those technical questions that most of our customers were asking, and to ensure that we reduce that waiting time. With this idea, my greatest question was: would it be viable? I can say, as for now, the system is not perfect yet and it has its limitations, but it has shown me that this concept is not only practical but it can be built, especially using current tools and with a lower price that small or medium enterprises could afford.

Another thing that I wanted to solve is the idea of localizing payments. Because we can't just copy a payment method in the Global North and take it to the Global South; something that may work well in Debrecen, Hungary, may not work well for users in Nairobi, Kenya. So, we needed to localize our payment and to ensure that we get the right payment method for the locality.

Basically, the goal of this thesis was to ensure that I answer two questions. The first one: is it really possible to combine Blazor and a RAG system together with a mobile payment that would work

well for a certain locality like M-Pesa? And the second one: is this project viable for future researchers? Does it clearly attract improvement, and if so, is it viable for other researchers to be able to collect user data, automate my idea, and test my idea on a large scale?

Bibliography

The following sources were cited in this thesis.

Jha, V. N., Anto, J., & KK, A. (2026). *Performance Comparison of Video-Based, Graph-Based, and Cloud-Hosted Vector Storage Systems: An Empirical Study of MemVid, Qdrant, and Amazon S3 Vector*. *International Journal of Scientific Research & Engineering Trends*, 12(1). <https://ijsret.com/>

Kolawole, O., Muritala, T. A., Akande, J. O., & Adekunle, A. O. (2024). *Digital Financial Services and the Performance of the Quoted Commercial Banks in Nigeria*. *International Journal of Professional Business Review*, 9(6), e04150. <https://doi.org/10.26668/businessreview/2024.v9i6.4150>

Nielsen, J. (2023). *AI in UX: Chatbots, agents, and the paradox of choice in digital retail*. Nielsen Norman Group. <https://www.nngroup.com/articles/ai-ux/>

Smith, A., & Jones, B. (2022). *Luxury brand engagement in the metaverse and digital storefronts*. *Journal of Digital Commerce*, 15(3), 112-128.

Wachira, V. K., Kalui, F. M., & Gathii, J. (2021). *Relationship between bank innovations and financial performance of commercial banks in Kenya*. *International Journal of Education and Research*, 3(5), 443-456. <https://www.ijern.com/journal/2015/May-2015/37.pdf>

McKinsey & Company. (2024). *The state of luxury e-commerce: Gen Z, digital natives, and the new rules of brand engagement*. McKinsey Global Institute. <https://www.mckinsey.com/industries/retail/our-insights/the-state-of-fashion-2024-finding-pockets-of-growth>

Microsoft. (2024a). *ASP.NET Core Blazor documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/blazor/>

Microsoft. (2024b). *Semantic Kernel SDK documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/semantic-kernel/overview/>

OpenAI. (2024a). *Embeddings guide.* OpenAI Platform Documentation. <https://platform.openai.com/docs/guides/embeddings>

OpenAI. (2024b). *GPT-4o mini: Advancing cost-efficient intelligence.* OpenAI Blog. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>

Qdrant. (2024). *Qdrant documentation: Collections, points, and HNSW indexing.* Qdrant Vector Database. <https://qdrant.tech/documentation/>

Safaricom. (2024). *Daraja API 2.0 developer guide: STK Push and callback specification.* Safaricom Developer Portal. <https://developer.safaricom.co.ke/>

DECLARATION ON THE USE OF AI

I, Tanui Kipngetich Sila, a Master's student in Computer Science at the Faculty of Computer Science, University of Debrecen, declare that this work, entitled AI-DRIVEN E-COMMERCE AUTOMATION USING BLAZOR WEB FRAMEWORK, is my own work.

I confirm that all sources used in the preparation of this work have been properly cited and cited in accordance with academic standards. Any ideas, data or text borrowed from other authors have been clearly indicated. I also declare that this work, in whole or in part, has not been submitted for any other degree or qualification at this or any other institution.

I wanted to explain how I used AI to create the Tanui watch store and ensure that I am transparent about this process. From the very beginning, I used these tools as a kind of creative assistant to figure out what was really trendy in the watch market and how to make the whole project coherent. I actually spent a lot of time brainstorming ideas with models like GPT-5.0, Gemini, and Claude Opus to make sure the user journey flow wasn't confusing and the ideas felt cohesive.

When it came time to actually build it, I decided to let the AI do some pretty standard, but admittedly time-consuming, tasks from scratch. I'm talking about things like login and registration pages or basic account setup. I have to admit that I used GPT models, including GPT 5.0, and the latest versions of Gemini and Claude to build the core code. By doing it this way, I didn't have to waste weeks on standard stuff and was able to focus my energy on the more complex parts of the project that I'm really proud of, like the MPESA payment integration and the RAG system. Speaking of this RAG setup, I want to emphasize how much I relied on the latest OpenAI models to handle the embedding and speech processing, which is why the search and store assistant feel so fast.

I also used these AI models as a high-level reviewer throughout the development process. Every time I finished a piece of handwritten code, I would go over it with them to see if I had missed anything obvious or if there was a clearer way to write a particular feature. Finally, I wanted to make sure that the work was still truly mine, so I used academix.cx to check and grade the project. It's an AI model that evaluates work to determine whether it was done by a human or an AI, and it confirmed that the essence of the project was original, with a very low percentage of AI markups.

This basically proved that while I was using these tools to work faster and smarter, Tanui Watch Store is still largely my own original work.